Github

inky4832@daum.net

Github 개요

1. Github 개요

Github 용도

git 원격 저장소를 제공하는 대표적인 서비스가 Github이다. 단순히 원격 저장소만을 제공하는 것이 아니라 여러가지 프로젝트 진행을 원활하게 하는 도구를 함께 제공한다.

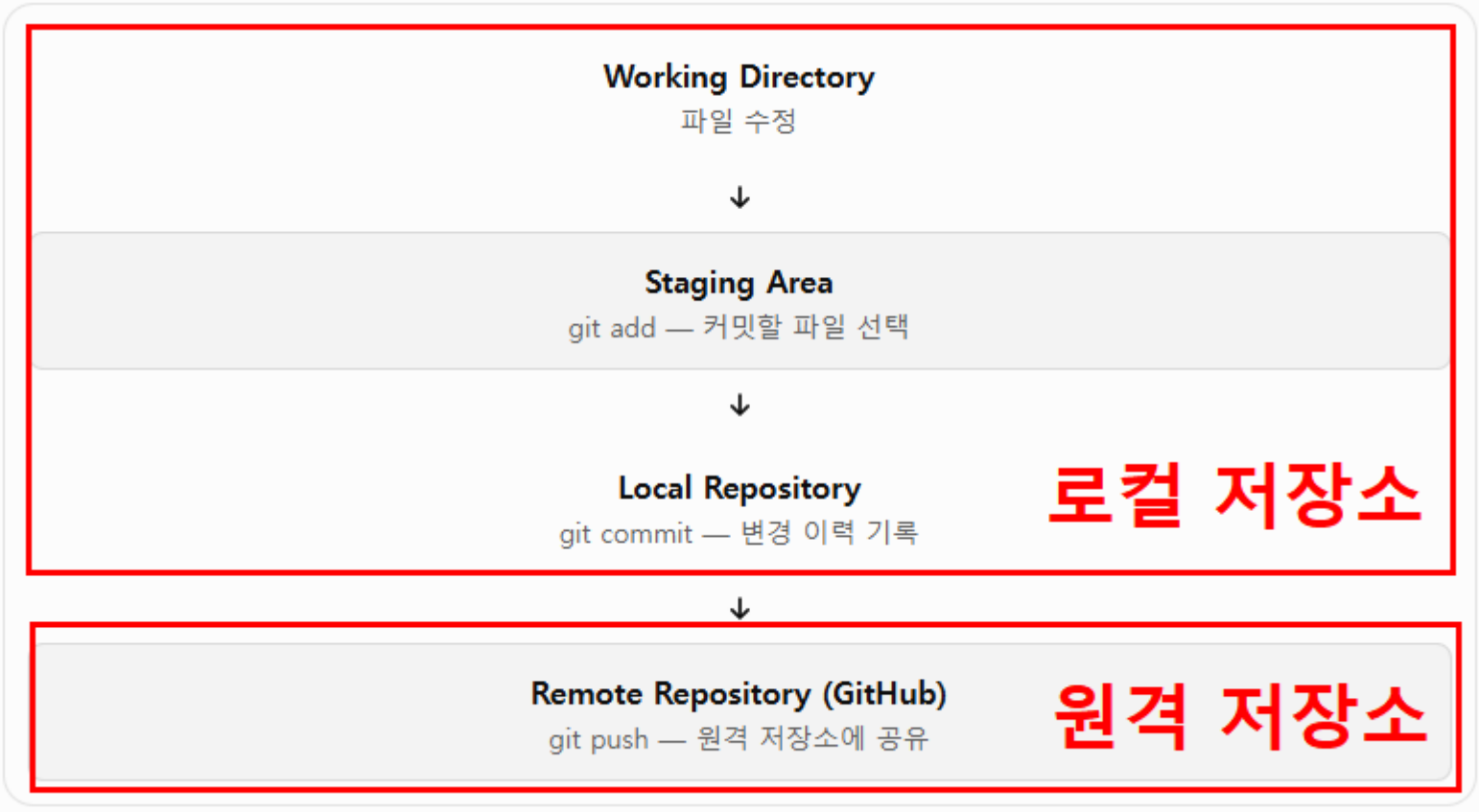
주요 기능

cloud에 git 저장소 제공 (push/pull , public/private)
code 관리 및 협업개발 가능 (issue, projects, Pull Request(PR))
automations & CI/CD (Github Actions 이용)

Github 장점

전 세계에서 진행되는 오픈 소스 프로젝트가 많이 모여 있어서 이에 참여하고 오픈소스에 기여할 수 있는 기회가 제공된다.
개발자는 Github를 이용해서 자신이 작성했던 코드 그 자체를 곧바로 제공할 수 있다.
개발 협업이 가능하다.

Github 작업흐름


Working Directory
파일 수정
Staging Area
git add — 커밋할 파일 선택
Local Repository
git commit — 변경 이력 기록
로컬 저장소
Remote Repository (GitHub)
git push — 원격 저장소에 공유
원격 저장소

2. 기본 명령어

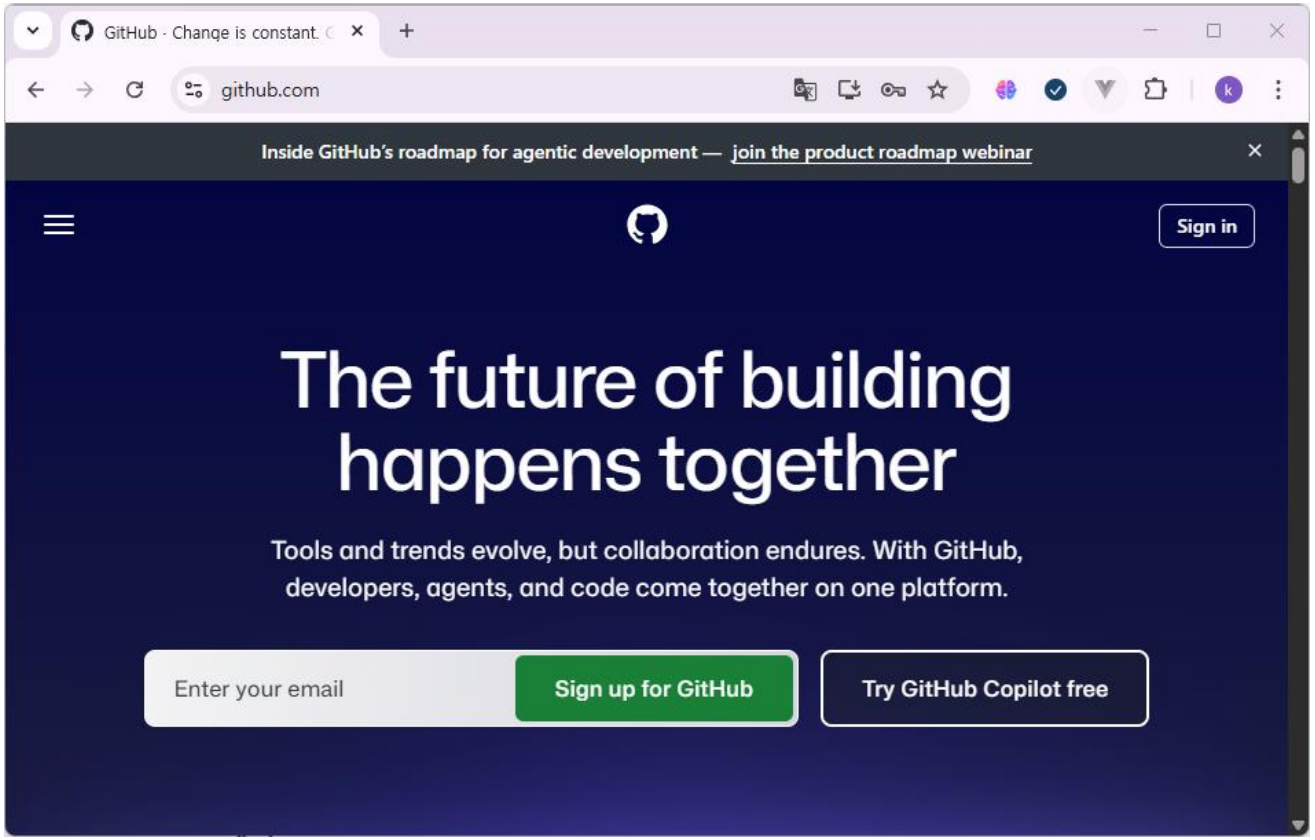
명령어	기능
git clone	원격 저장소의 모든 내용을 로컬 저장소로 복사한다.
git remote	로컬 저장소를 특정 원격 저장소와 연결한다.
git push	로컬 저장소의 커밋 내역을 원격 저장소로 보낸다.
git fetch	로컬 저장소와 원격 저장소의 커밋 버전이 다를 때, 원격 저장소의 커밋 내역을 로컬로 가져온다. 필요시 git merge 로 나중에 병합할 수 있다.
git pull	원격 저장소의 커밋 내역을 로컬로 가져와서 자동으로 병합한다. git fetch + git merge 기능
git branch –r git branch –a	원격 브랜치 목록 보기 로컬 및 원격 브랜치 목록 보기
git push origin --delete <브랜치명>	원격 브랜치 삭제
git fetch –p origin	로컬의 원격 브랜치 정보와 실제 원격 브랜치 정보 동기화. 예> 원격에서 branch 삭제 되어 실제 원격 branch와 로컬에서 보는 원격 branch가 다를 때 등.

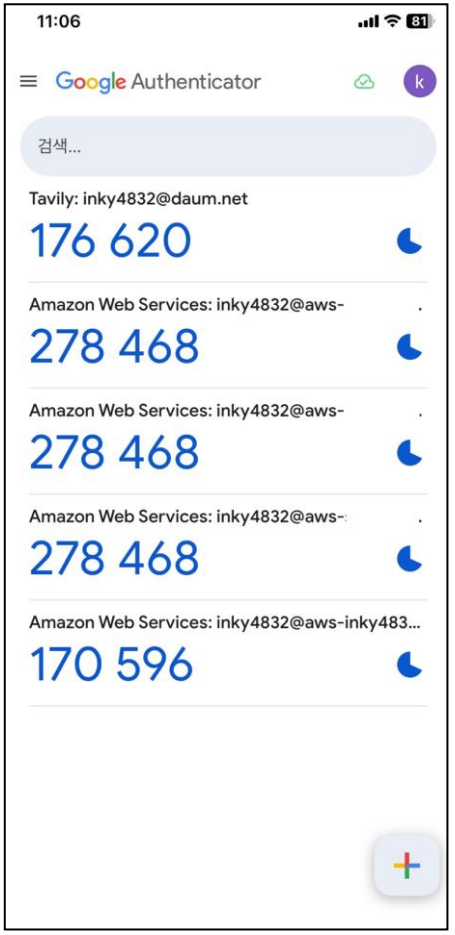
Github에 저장소 생성

1. Github 가입

Github 가입 및 로그인

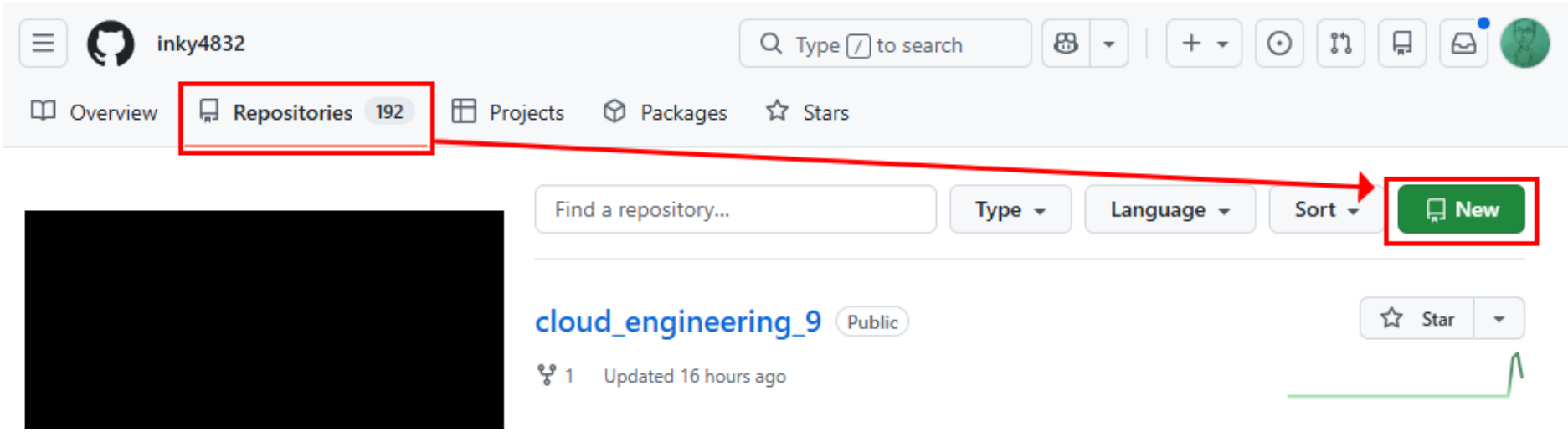
https://github.com/





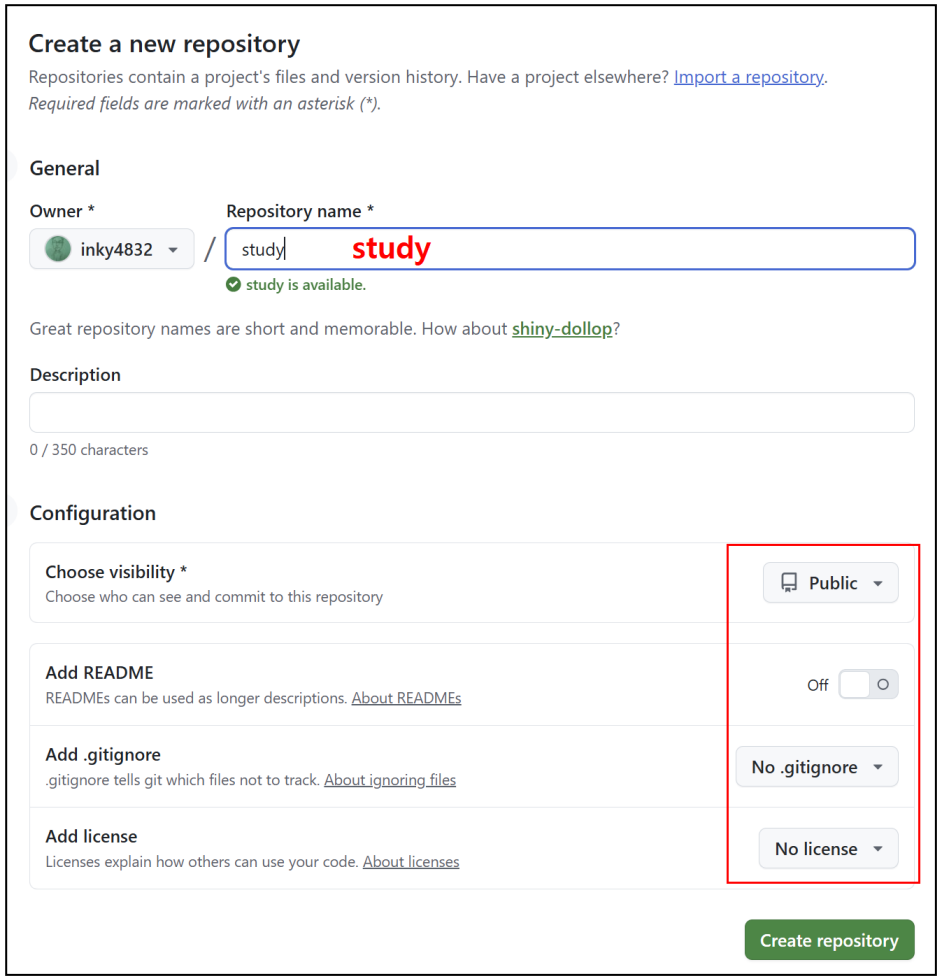
2. 원격 저장소 생성

Repositories 선택



2. 원격 저장소 생성

빈 원격 저장소 생성





clone

1. clone

개요

github에 생성된 원격 저장소를 로컬에 복제하는 방법이다.
clone 하면 로컬 컴퓨터에 새로운 디렉터리가 생성되고 초기화까지 된다.(git init 포함)
즉, 원격 저장소를 이용해서 로컬 저장소를 만드는 방법이다.

문법: git clone https://~ [로컬저장소명]

[로컬저장소명]은 생략가능. 생략하면 원격 저장소명으로 로컬저장소가 생김.

예> git clone https://github.com/inky4832/study.git [git-demo]

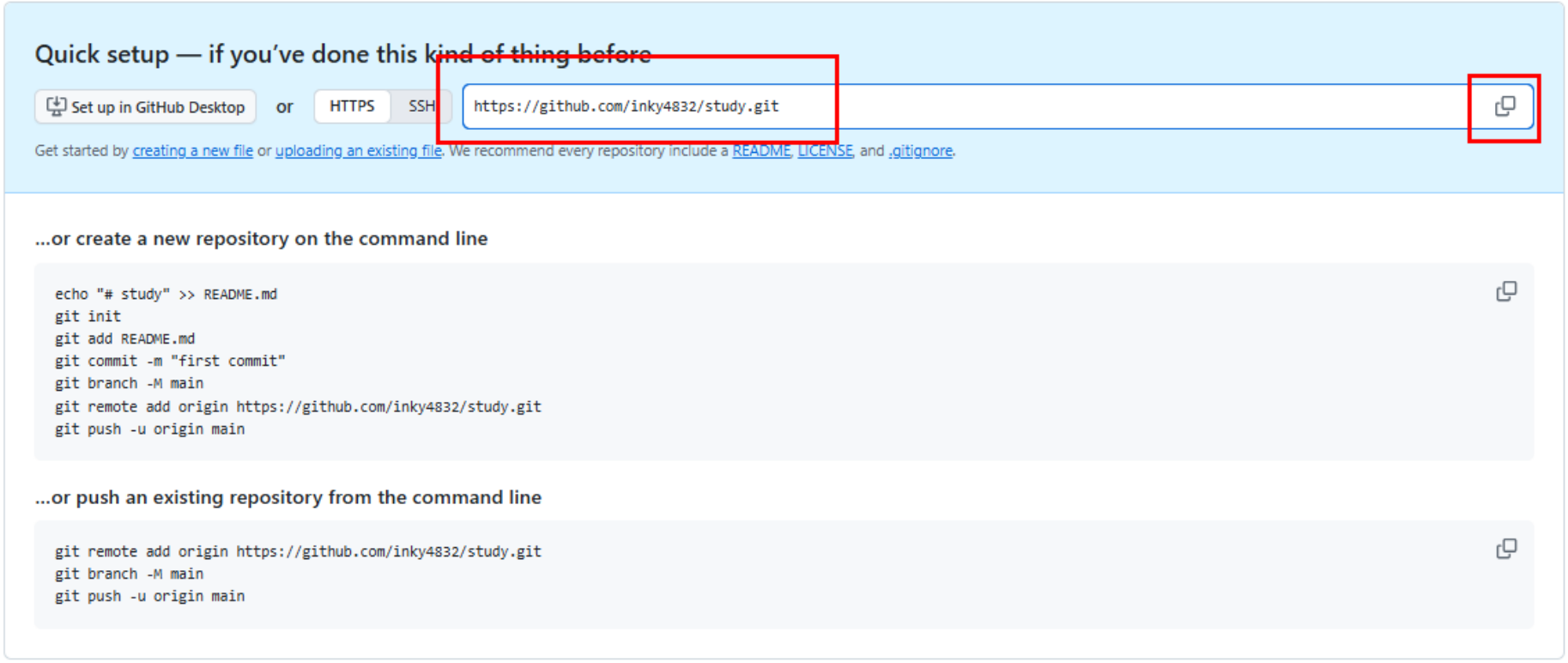
특징

만약 원격에 main 브랜치를 제외한 여러 개의 브랜치가 있다고 가정했을 때 clone 하더라도 모든 원격 브랜치를 가져오지 않는다.
main 브랜치만 자동으로 추적하고 나머지는 checkout + switch 해야 로컬에 브랜치가 생성된다.

2. clone 실습

가. 원격 저장소의 URL 복사

https://github.com/inky4832/study.git



2. clone 실습

나. git clone 명령어 실행

```
$ git clone https://github.com/inky4832/study.git
Cloning into 'study'...
warning: You appear to have cloned an empty repository.

User@DESKTOP-N2ID8DK UCRT64 /c/github_study
$ cd study/

User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ ls -al
total 4
drwxr-xr-x 1 User 197121 0 Oct 21 15:20 ./
drwxr-xr-x 1 User 197121 0 Oct 21 15:20 ../
drwxr-xr-x 1 User 197121 0 Oct 21 15:20 .git/

User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)

User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
```

로컬 저장소 생성

다. 원격 저장소 연결 상태 확인

문법: git remote
git remote -v

https://git-scm.com/docs/git-remote

```
$ git remote
origin

User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git remote -v
origin  https://github.com/inky4832/study.git (fetch)
origin  https://github.com/inky4832/study.git (push)
```

라. 원격 저장소 연결 종료

문법: git remote remove origin
git remote rm origin

```
User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git remote remove origin

User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git remote -v

User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$
```

2. clone 실습

마. 로컬 저장소와 원격 저장소 연결

문법: git remote add origin https://~

```
User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git remote add origin https://github.com/inky4832/study.git

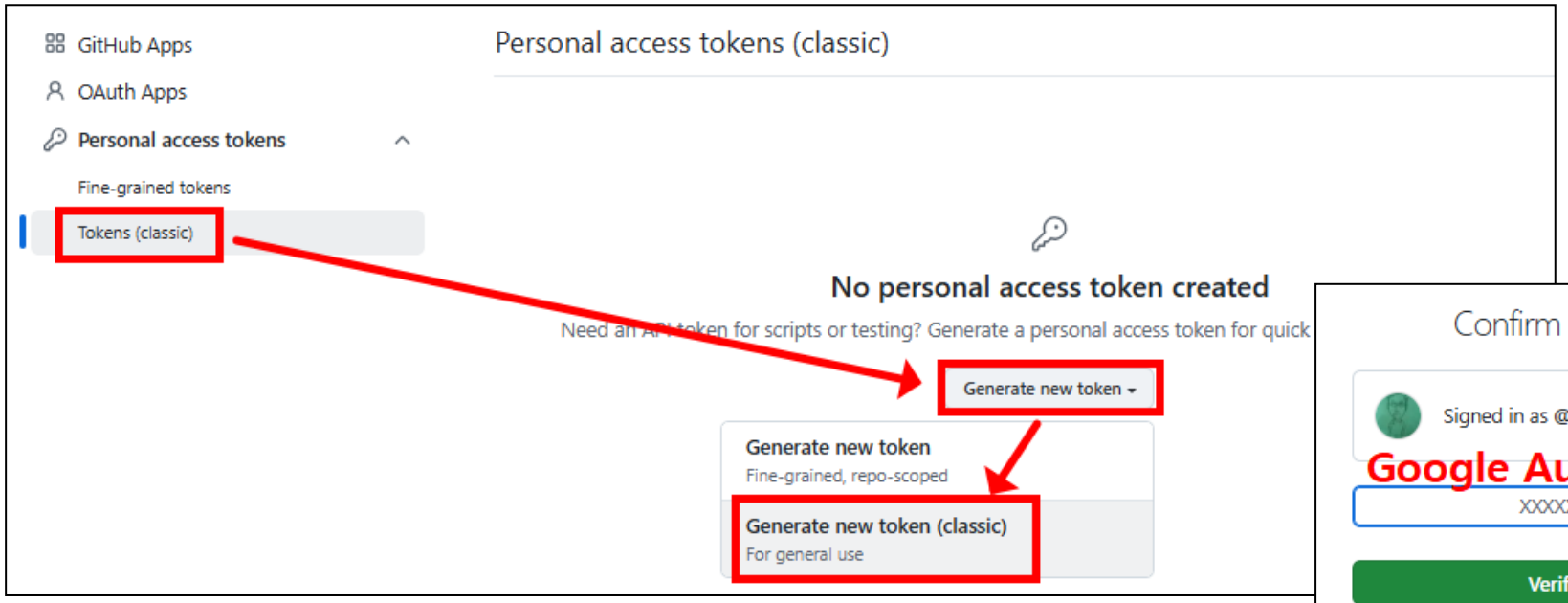
User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git remote -v
origin  https://github.com/inky4832/study.git (fetch)
origin  https://github.com/inky4832/study.git (push)
```

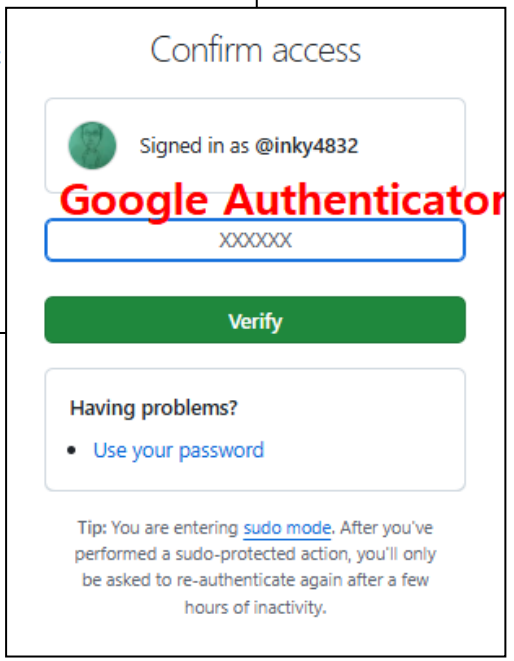
로컬 저장소의 **main** 브랜치를
원격 저장소에 올리기

1. main 브랜치를 원격 저장소에 올리기

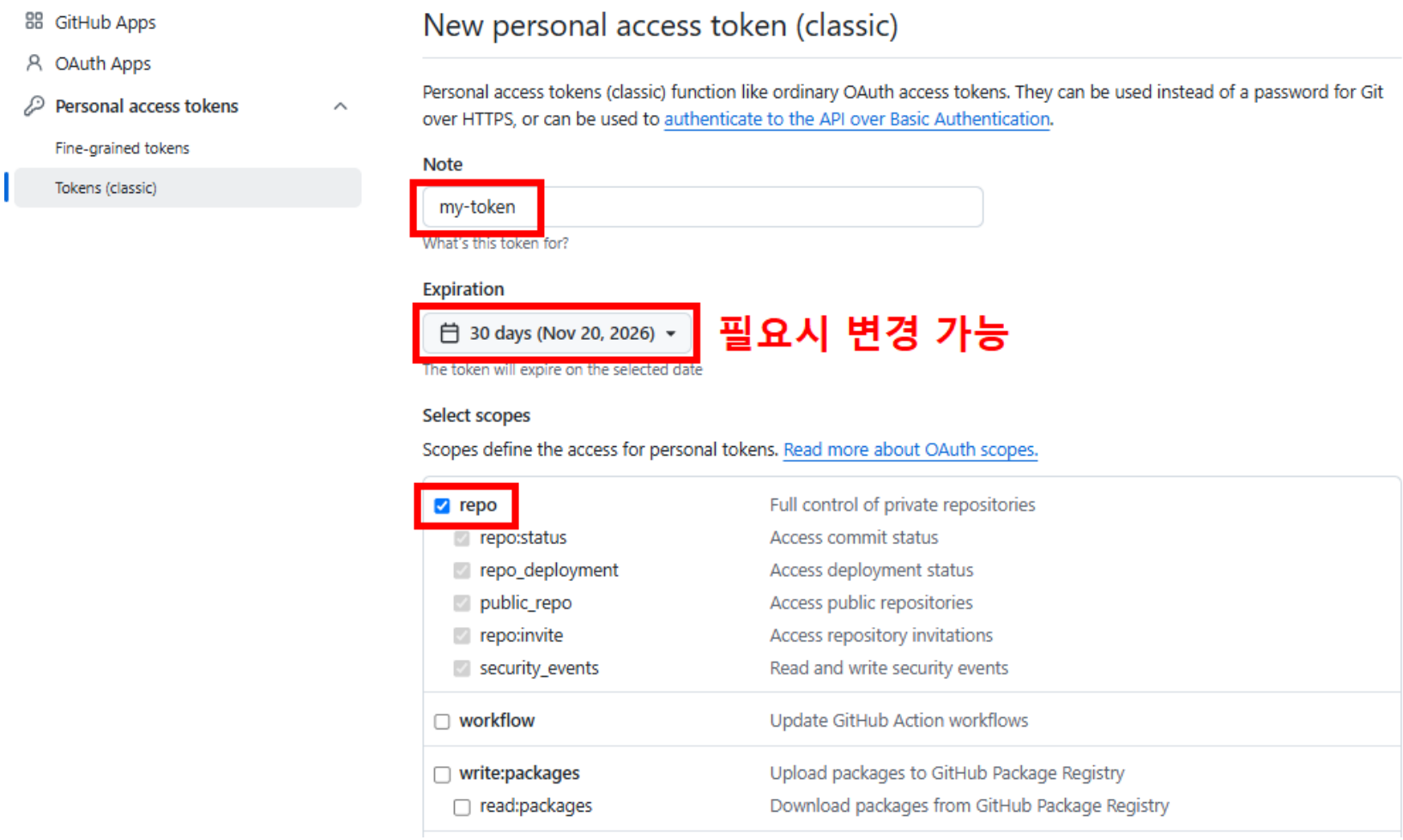
1. token 얻기

https://github.com/settings/tokens

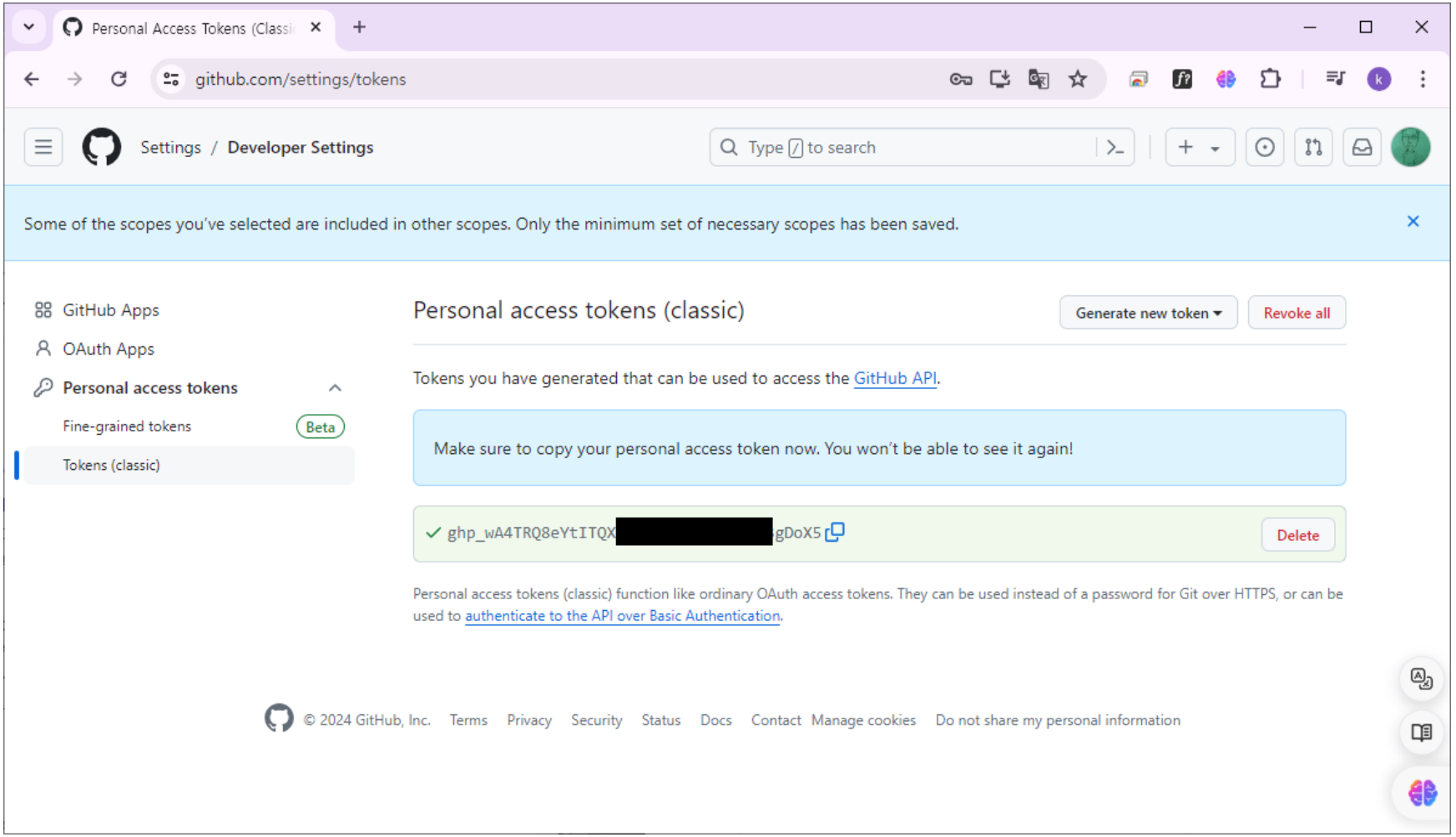




1. main 브랜치를 원격 저장소에 올리기

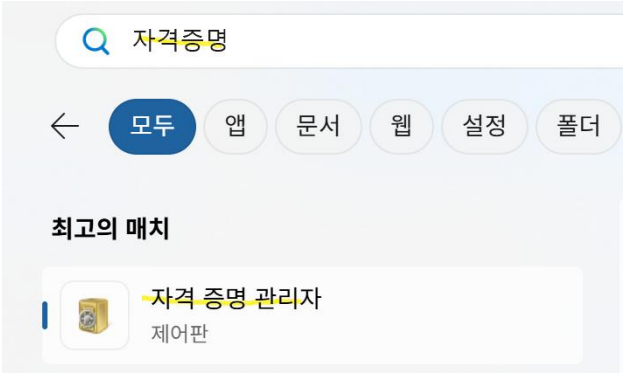


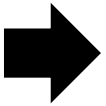
1. main 브랜치를 원격 저장소에 올리기



1. main 브랜치를 원격 저장소에 올리기

github 인증자격 제거하기







1. main 브랜치를 원격 저장소에 올리기

2. 로컬 커밋 내역을 원격 저장소에 올리기

문법: git push -u origin main

https://git-scm.com/docs/git-push

로컬 저장소 커밋

```
$ touch hello.txt
$ echo "hello" >> hello.txt
$ cat hello.txt
  hello
$ git add hello.txt
$ git commit -m “hello commit”
```

원격 저장소 연결 및 확인

```
$ git remote add origin https://github.com/inky4832/study.git
$ git remote -v
```

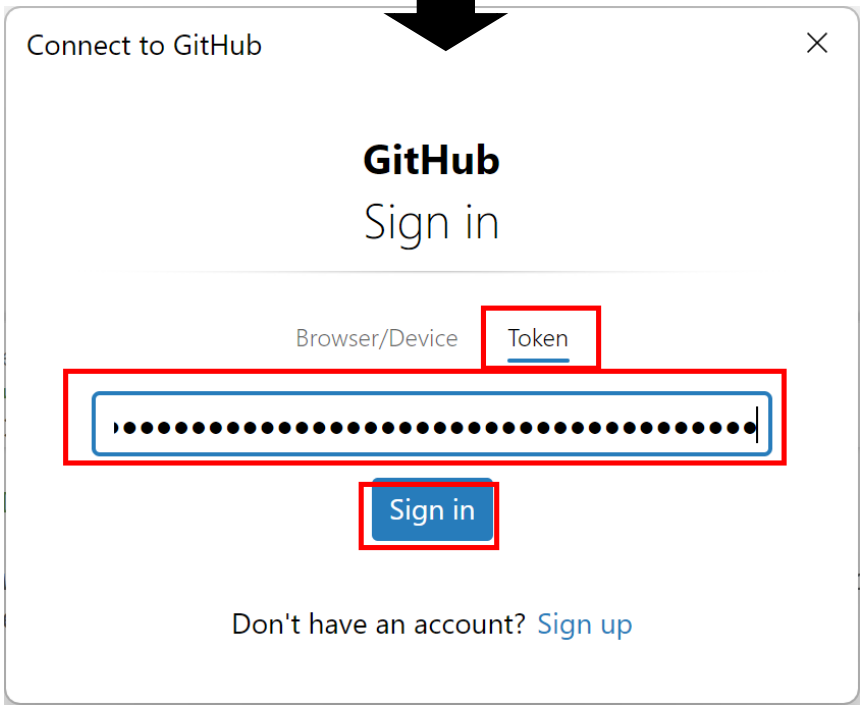
1. main 브랜치를 원격 저장소에 올리기

원격 저장소에 push

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git status
On branch main
nothing to commit, working tree clean
```

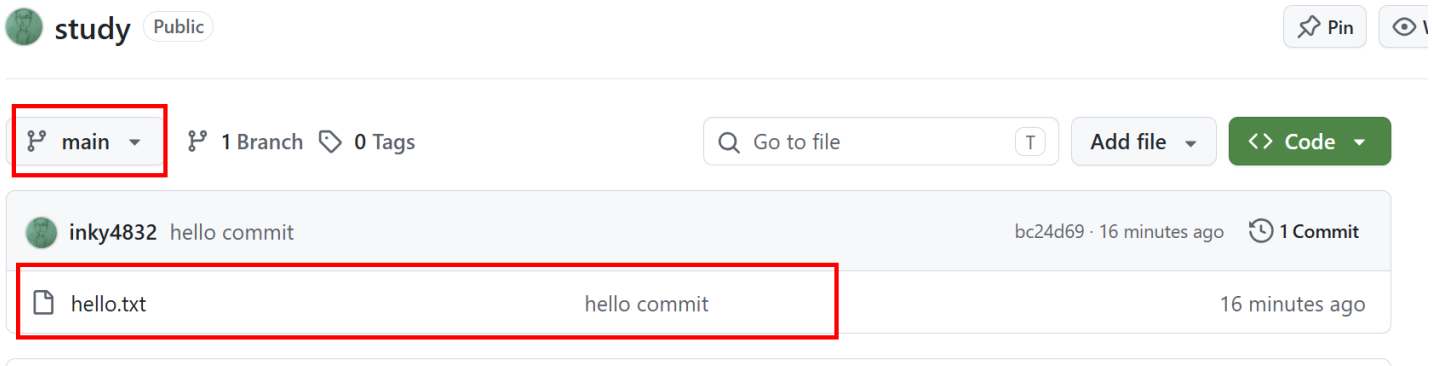
push 하기전에 반드시 status로 확인하여 아무런 변경사항이 없어야 된다. 있다면 commit 먼저 작업해야 된다.

```
$ git push origin main
```



1. main 브랜치를 원격 저장소에 올리기


```
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 211 bytes | 211.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/inky4832/study.git
 * [new branch]      main -> main
```



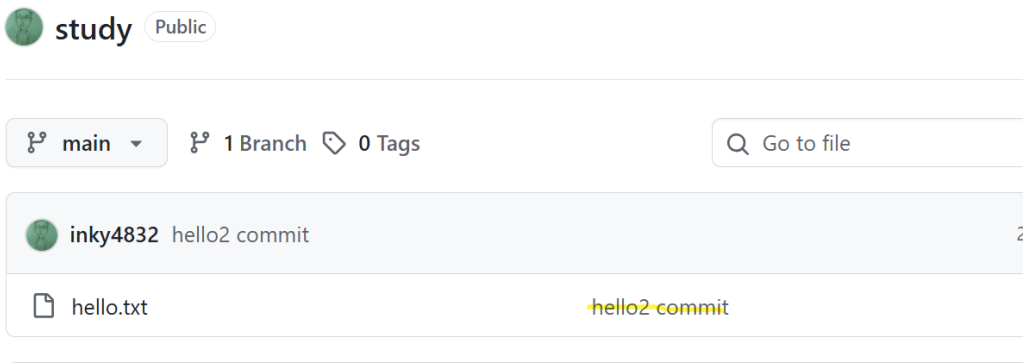
1. main 브랜치를 원격 저장소에 올리기

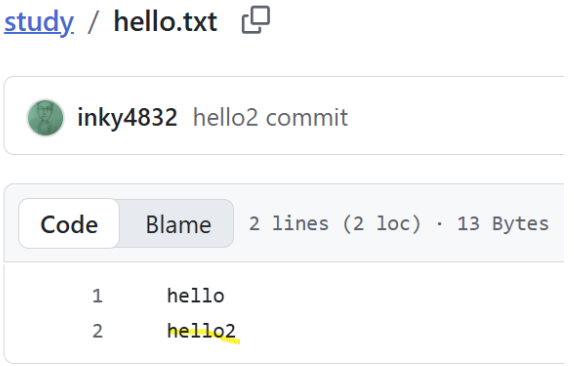
커밋 정보 보기

```
$ git log
commit bc24d6968f1a68f558841c9a96bf2c452ef25c01 (HEAD -> main, origin/main)
Author: inky4832 <inky4832@daum.net>
Date:   Wed Oct 21 16:20:39 2026 +0900

    hello commit
```

이후에 한번 더 hello.txt 파일 수정하고 "hello2 commit" 생성하고 push 한다.





로컬 저장소의 **hotfix** 브랜치를
원격 저장소에 올리기

1. hotfix 브랜치를 원격 저장소에 올리기

1. hotfix 브랜치 생성 및 커밋하기

문법: git push origin hotfix

https://git-scm.com/docs/git-push

hotfix 브랜치 생성 및 커밋

```
$ git branch hotfix
$ git switch hotfix
$ git branch
$ echo "hello3 hotfix" >> hello.txt
$ cat hello.txt
  hello1
  hello2
  hello3 hotfix
$ git commit -am “hello3 hotfix commit”
```

원격 저장소 연결 및 확인

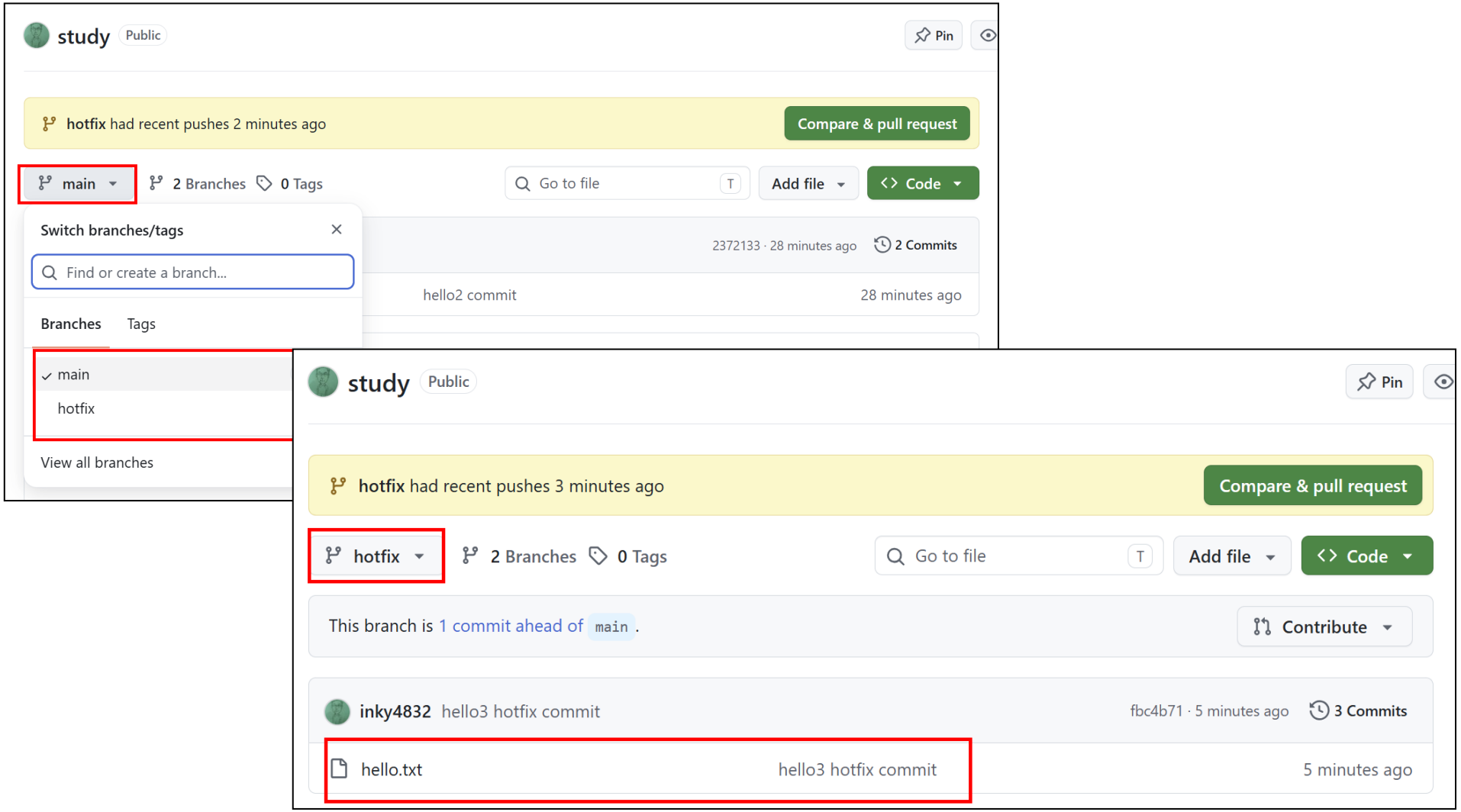
```
$ git remote add origin https://github.com/inky4832/study.git
$ git remote -v
```

2. hotfix 브랜치를 원격 저장소에 push

hotfix 브랜치 push 하기

```
$ git push origin hotfix
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 262 bytes | 262.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'hotfix' on GitHub by visiting:
remote:      https://github.com/inky4832/study/pull/new/hotfix
remote:
To https://github.com/inky4832/study.git
 * [new branch]      hotfix -> hotfix
```

2. hotfix 브랜치를 원격 저장소에 올리기



3. commit 정보보기

```
$ git log --oneline main
2372133 (origin/main, main) hello2 commit
bc24d69 hello commit
```

```
$ git log --oneline hotfix
fbc4b71 (HEAD -> hotfix, origin/hotfix) hello3 hotfix commit
2372133 (origin/main, main) hello2 commit
bc24d69 hello commit
```

```
$ git log --oneline origin/main
2372133 (origin/main, main) hello2 commit
bc24d69 hello commit
```

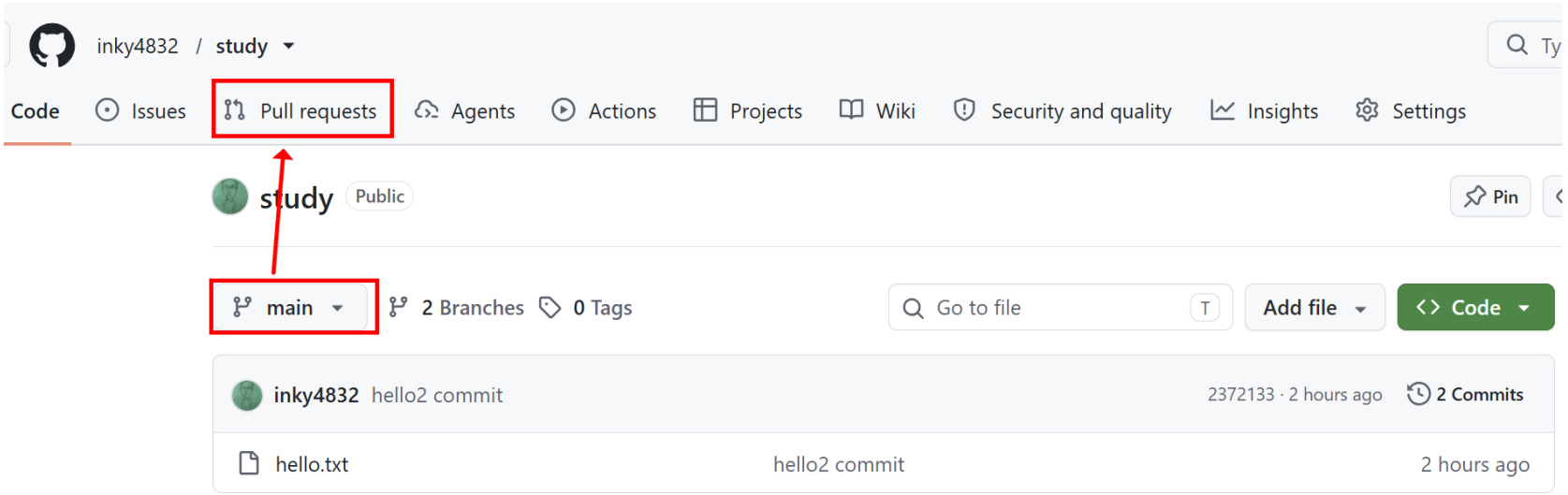
```
$ git log --oneline origin/hotfix
fbc4b71 (HEAD -> hotfix, origin/hotfix) hello3 hotfix commit
2372133 (origin/main, main) hello2 commit
bc24d69 hello commit
```

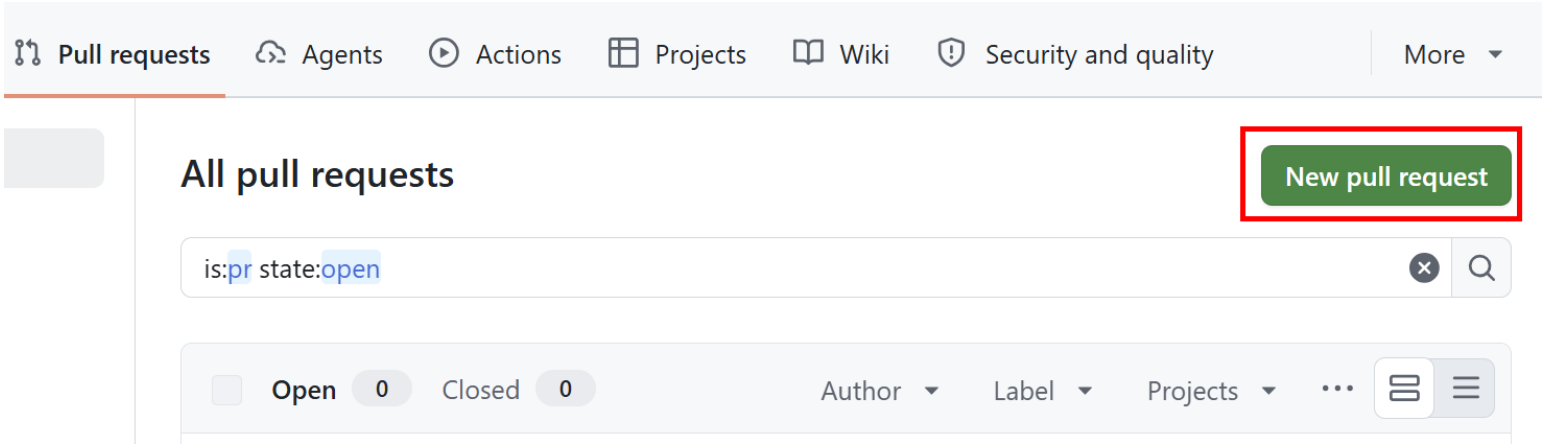
```
$ git log --oneline
fbc4b71 (HEAD -> hotfix, origin/hotfix) hello3 hotfix commit
2372133 (origin/main, main) hello2 commit
bc24d69 hello commit
```

원격 저장소에서 **hotfix** 병합하기 **(Pull Requests** 이용**)**

1. 원격 저장소에서 hotfix 병합하기

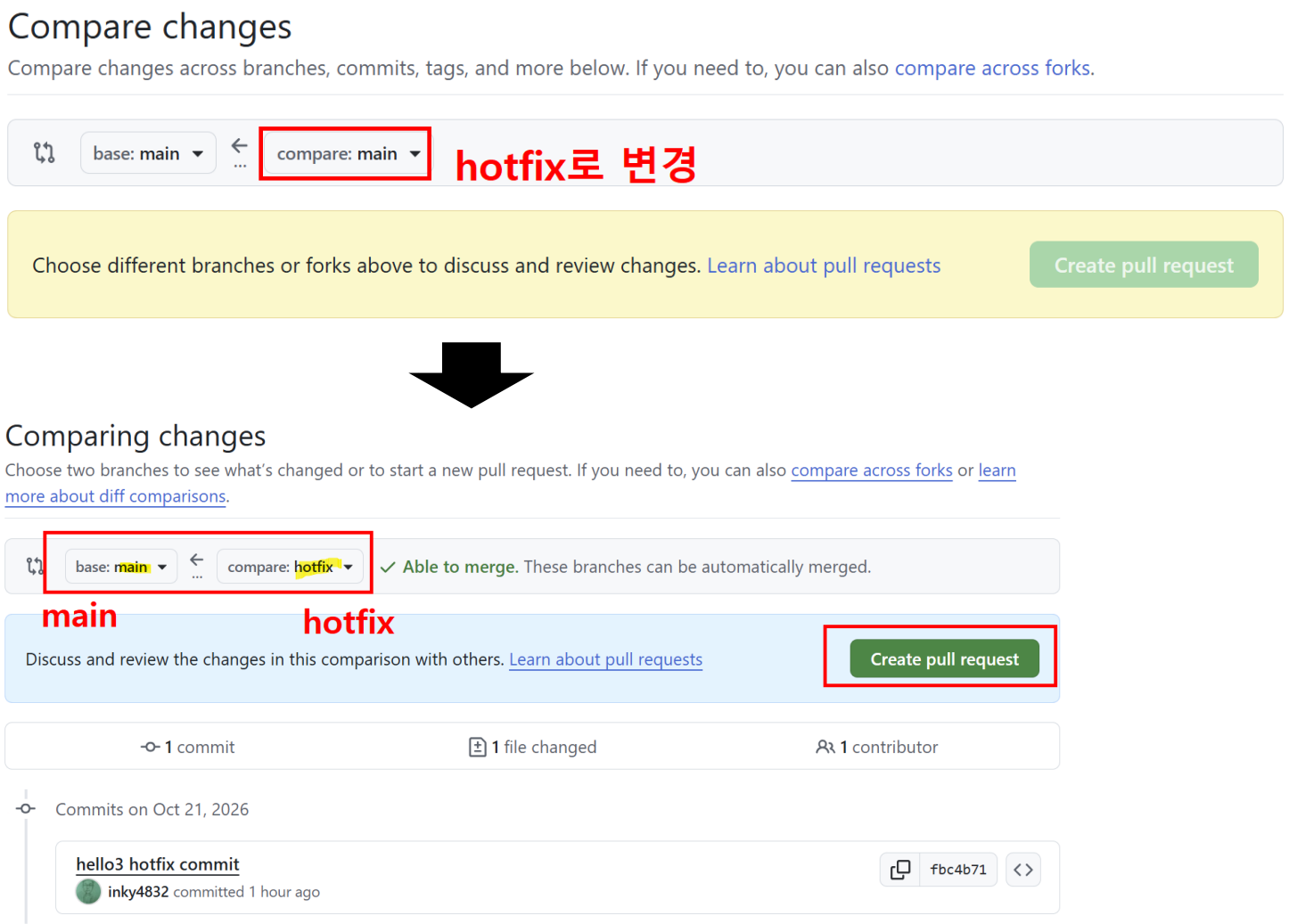
1. main 브랜치 선택하고 Pull Requests 선택





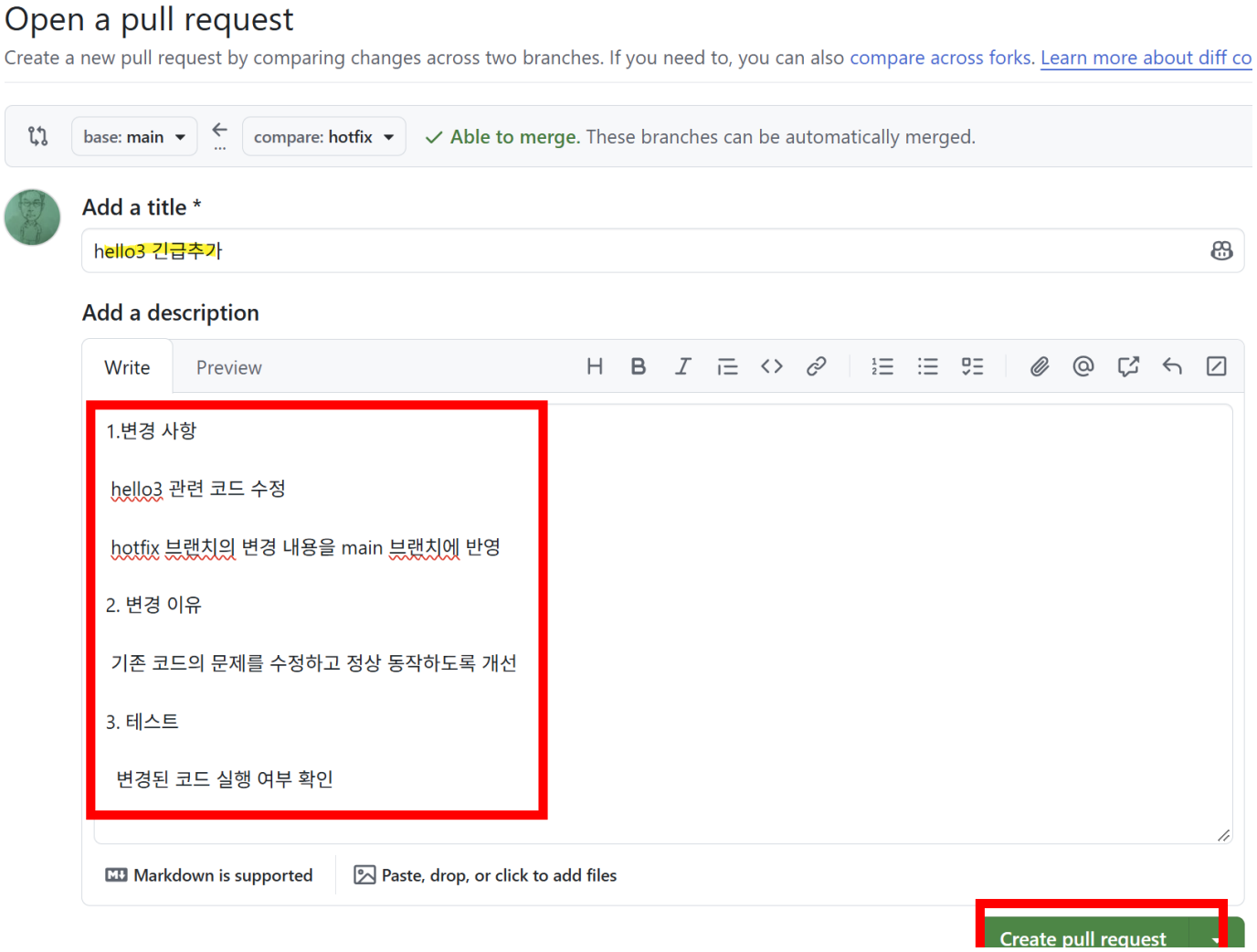
1. 원격 저장소에서 hotfix 병합하기

2. 병합할 타겟 hotfix 브랜치 선택하기, 코드리뷰



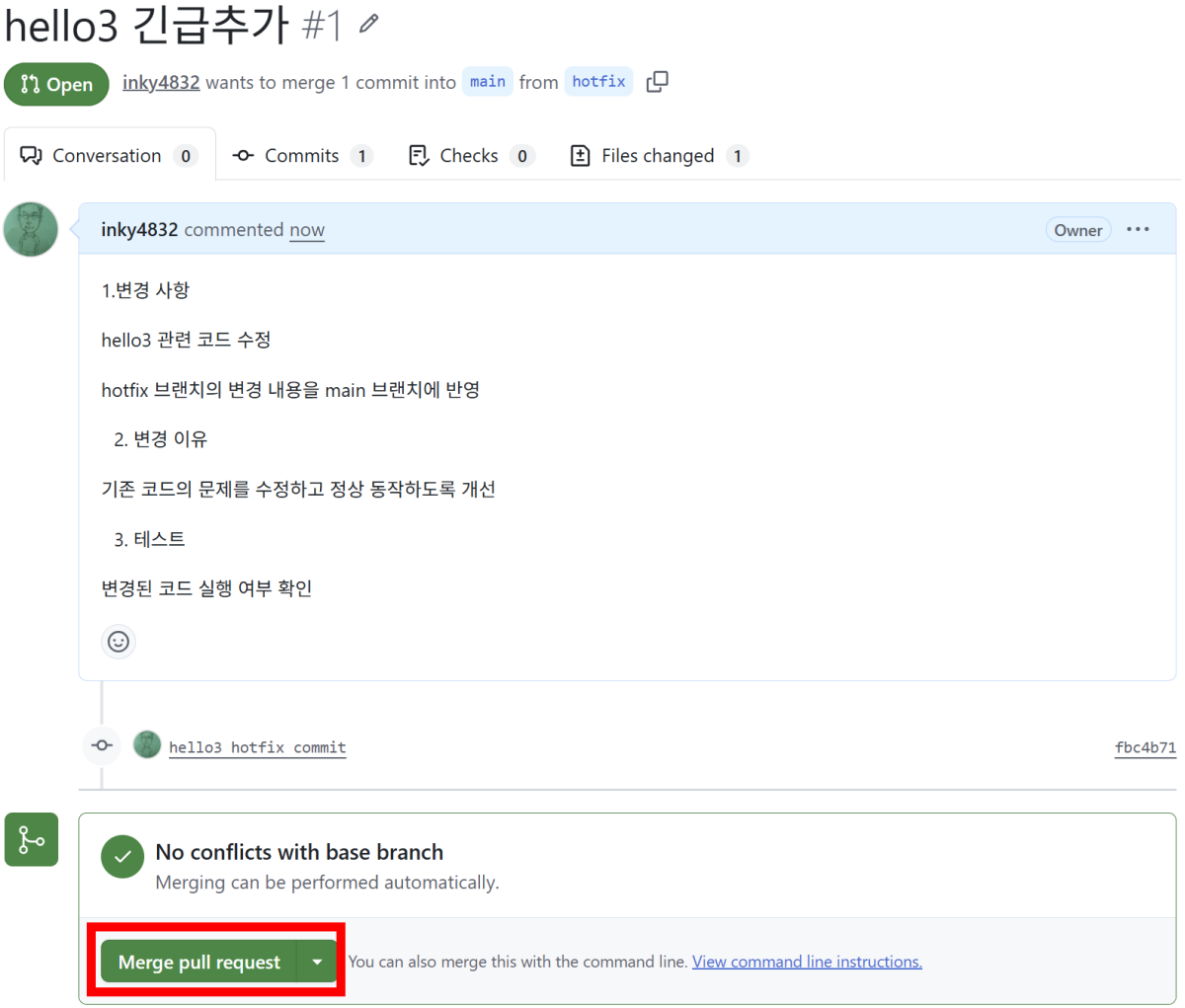
1. 원격 저장소에서 hotfix 병합하기

3. PR 설명 추가

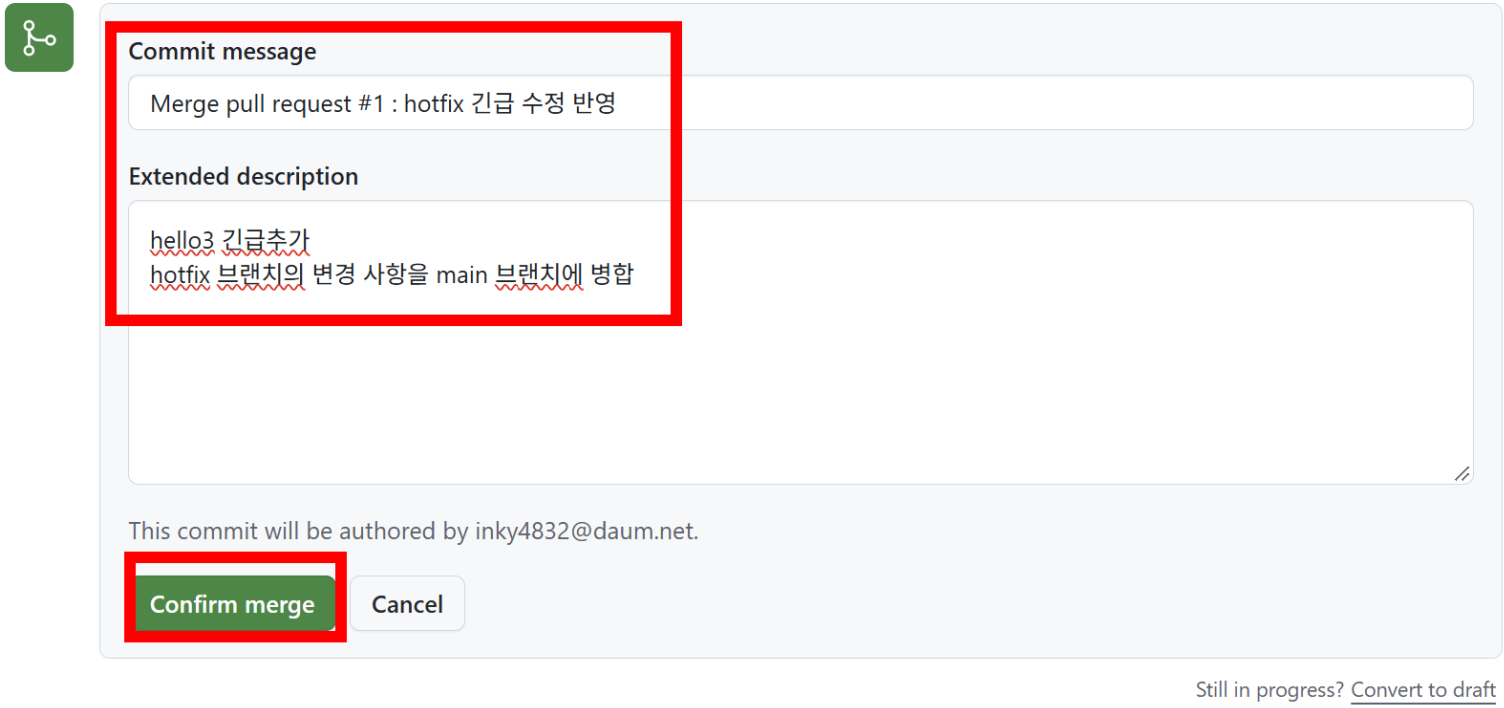


1. 원격 저장소에서 hotfix 병합하기

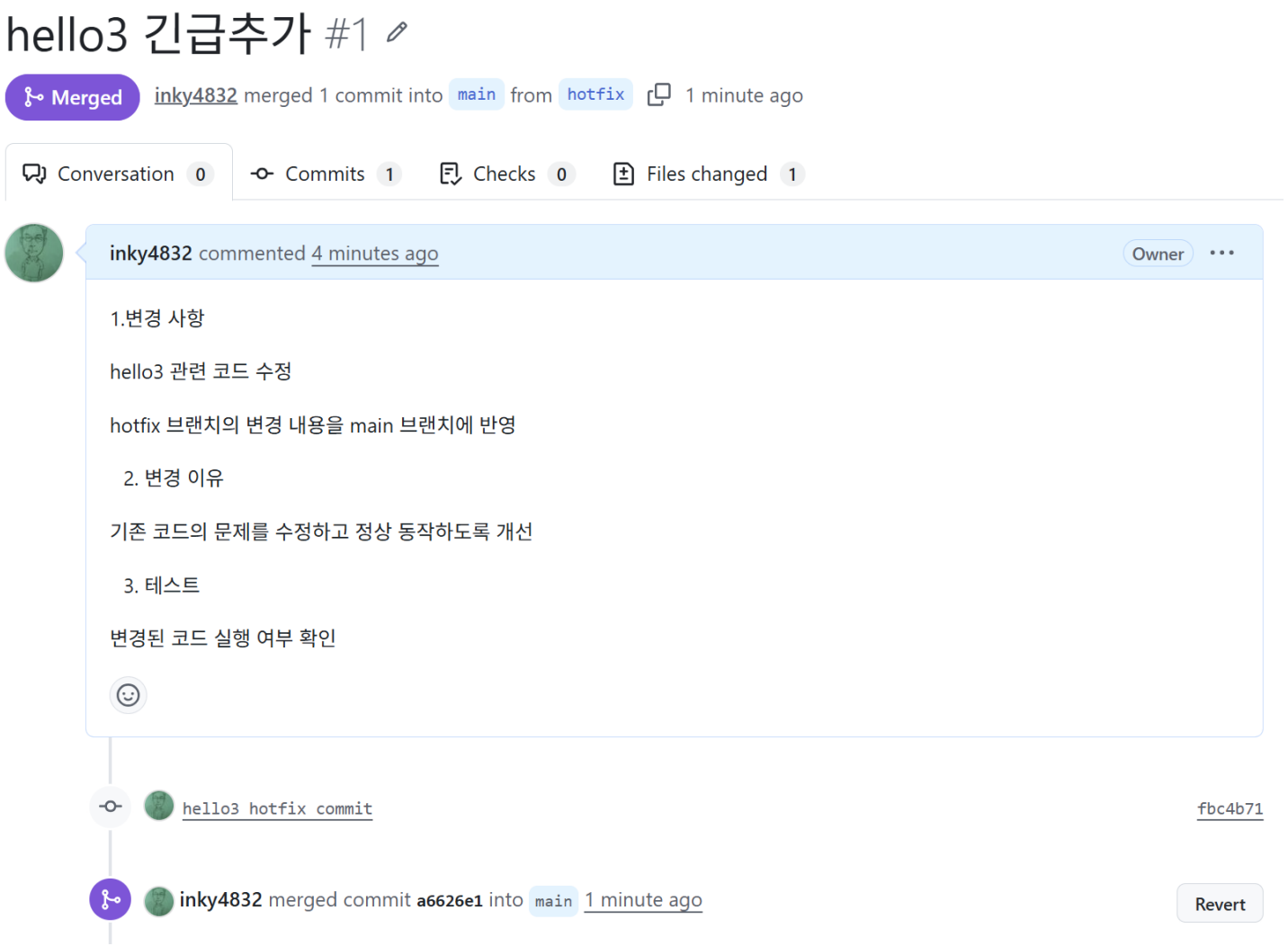
4. 병합하기



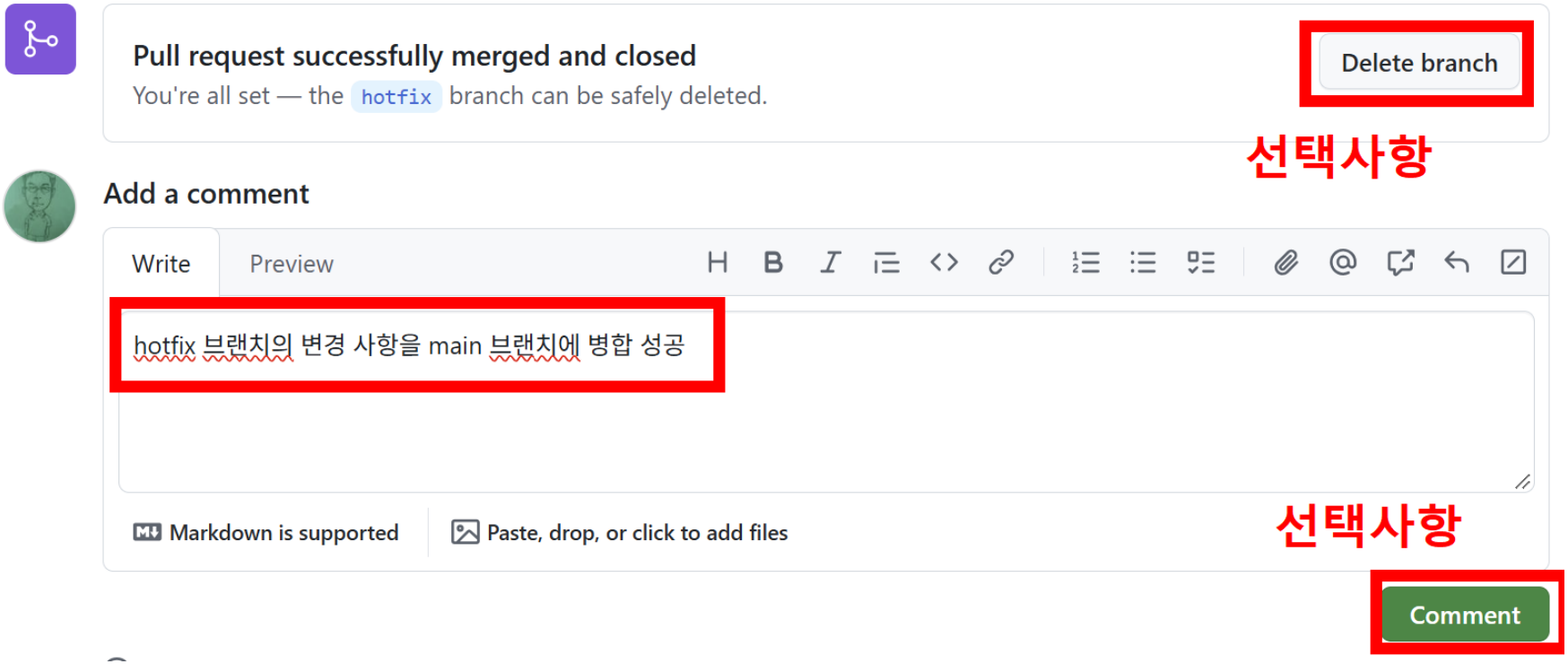
1. 원격 저장소에서 hotfix 병합하기



5. hotfix 브랜치 삭제 및 Comment 추가 (옵션)

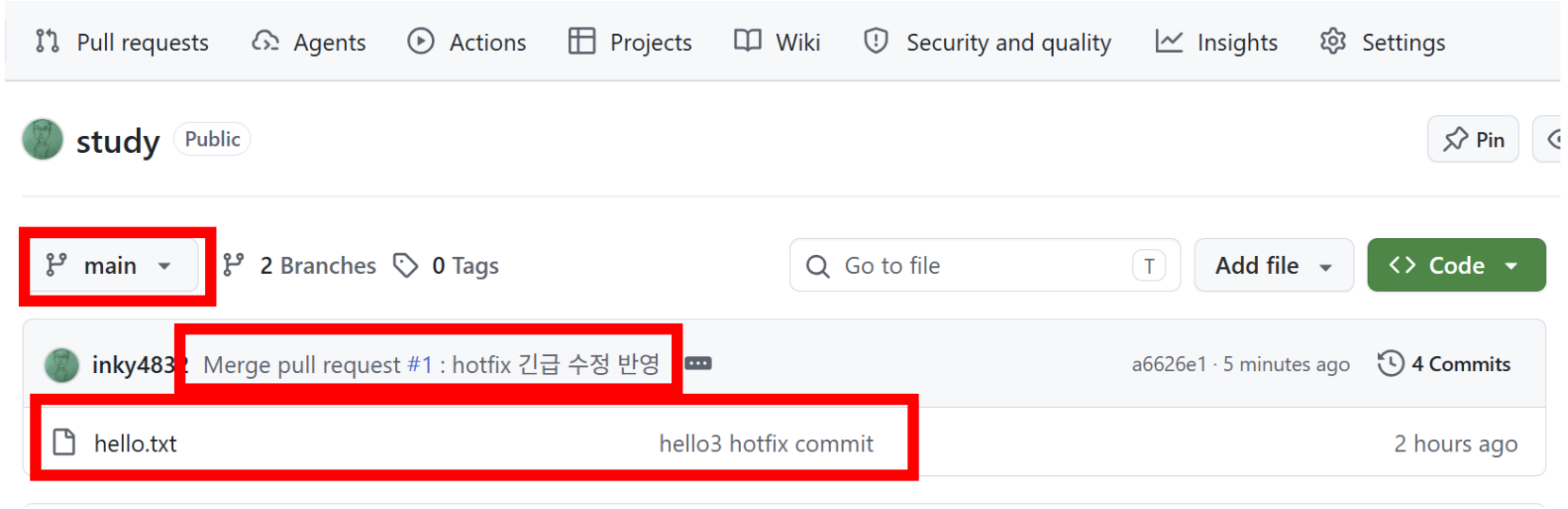


1. 원격 저장소에서 hotfix 병합하기

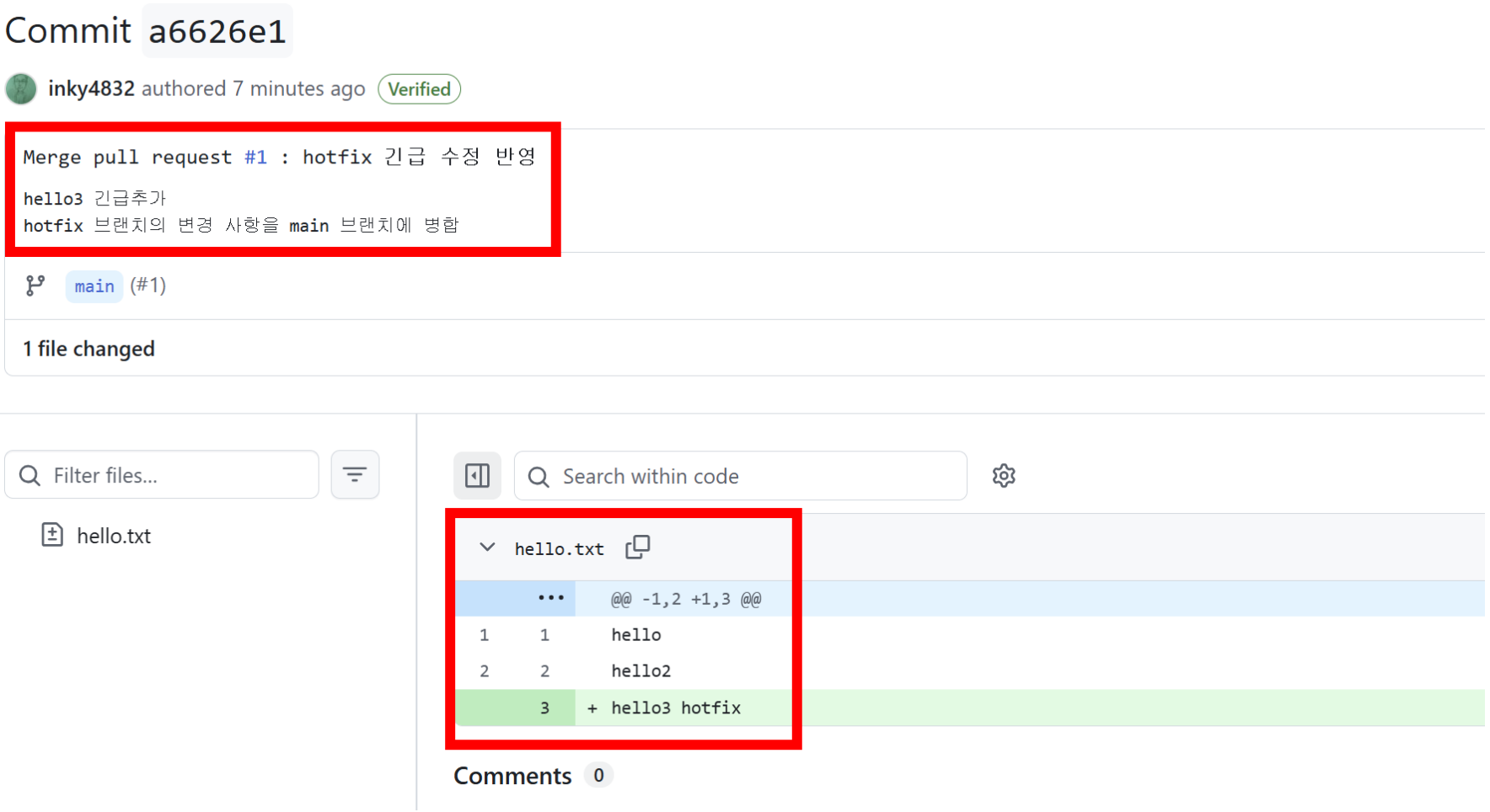


1. 원격 저장소에서 hotfix 병합하기

6. 병합 확인하기



1. 원격 저장소에서 hotfix 병합하기



1. 원격 저장소에서 hotfix 병합하기

7. 원격 저장소의 hotfix 브랜치 삭제하기

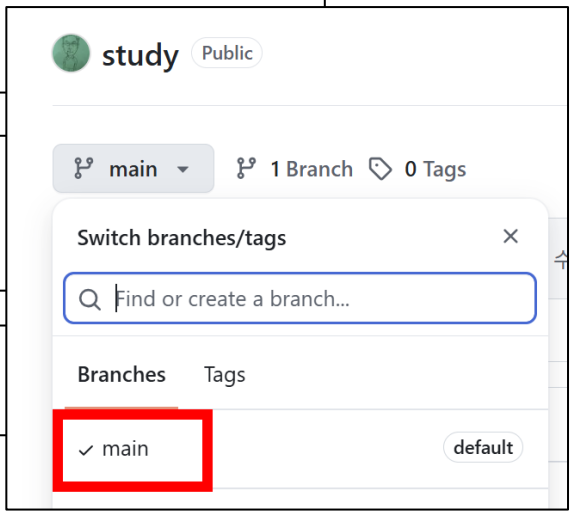
문법: git push origin --delete hotfix

```
$ git branch -r
  origin/hotfix
  origin/main
```

```
$ git branch -a
* hotfix
  main
  remotes/origin/hotfix
  remotes/origin/main
```

```
$ git push origin --delete hotfix
To https://github.com/inky4832/study.git
 - [deleted]         hotfix
```

```
$ git branch -r
  origin/main
```



원격 저장소의 **main** 브랜치를
로컬 저장소로 가져와서 병합하기

1. fetch + merge

현재 로컬 저장소 main 커밋 상태 확인

```
User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git log --oneline
2372133 (HEAD -> main, origin/main) hello2 commit
bc24d69 hello commit

User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ cat hello.txt
hello
hello2
```

현재 원격 저장소 main 커밋 상태 확인

```
User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git log --oneline origin/main
2372133 (HEAD -> main, origin/main) hello2 commit
bc24d69 hello commit
```

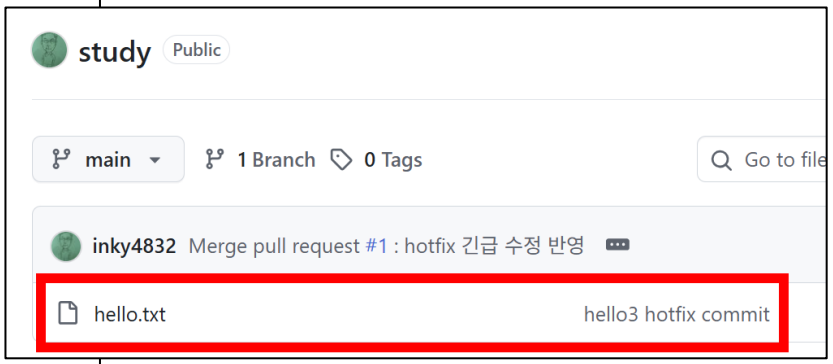
로컬 저장소에 있는 원격의 main 커밋 정보
(실제 원격 커밋정보와 일치 안될 수 있음)

```
User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git fetch origin
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 991 bytes | 52.00 KiB/s, done.
From https://github.com/inky4832/study
   2372133..a6626e1  main       -> origin/main
```

원격의 최신 정보를 로컬로 가져와서 반영

```
User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git log --oneline origin/main
a6626e1 (origin/main, origin/HEAD) Merge pull request #1 : hotfix 긴급 수정 반영
fbc4b71 (hotfix) hello3 hotfix commit
2372133 (HEAD -> main) hello2 commit
bc24d69 hello commit
```

로컬에 반영된 원격 최신정보 출력



1. fetch + merge

로컬 저장소와 원격 저장소 병합

문법: git fetch origin
git merge origin/main

```
User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git merge origin/main
Updating 2372133..a6626e1
Fast-forward
 hello.txt | 1 +
 1 file changed, 1 insertion(+)
```

현재 병합은 Fast-forward 임.
(병합커밋도 발생)

```
User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ git log --oneline
a6626e1 (HEAD -> main, origin/main, origin/HEAD) Merge pull
fbc4b71 (hotfix) hello3 hotfix commit
2372133 hello2 commit
bc24d69 hello commit

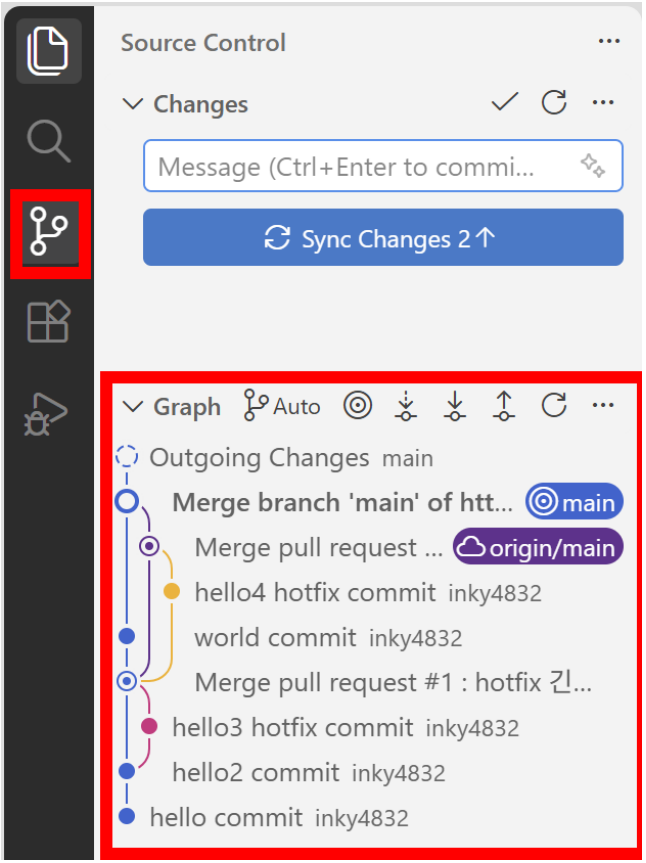
User@DESKTOP-N2ID8DK UCRT64 /c/github_study/study (main)
$ cat hello.txt
hello
hello2
hello3 hotfix
```

2. pull

로컬 저장소와 원격 저장소 병합

문법: git pull origin main

git pull = git fetch + git merge

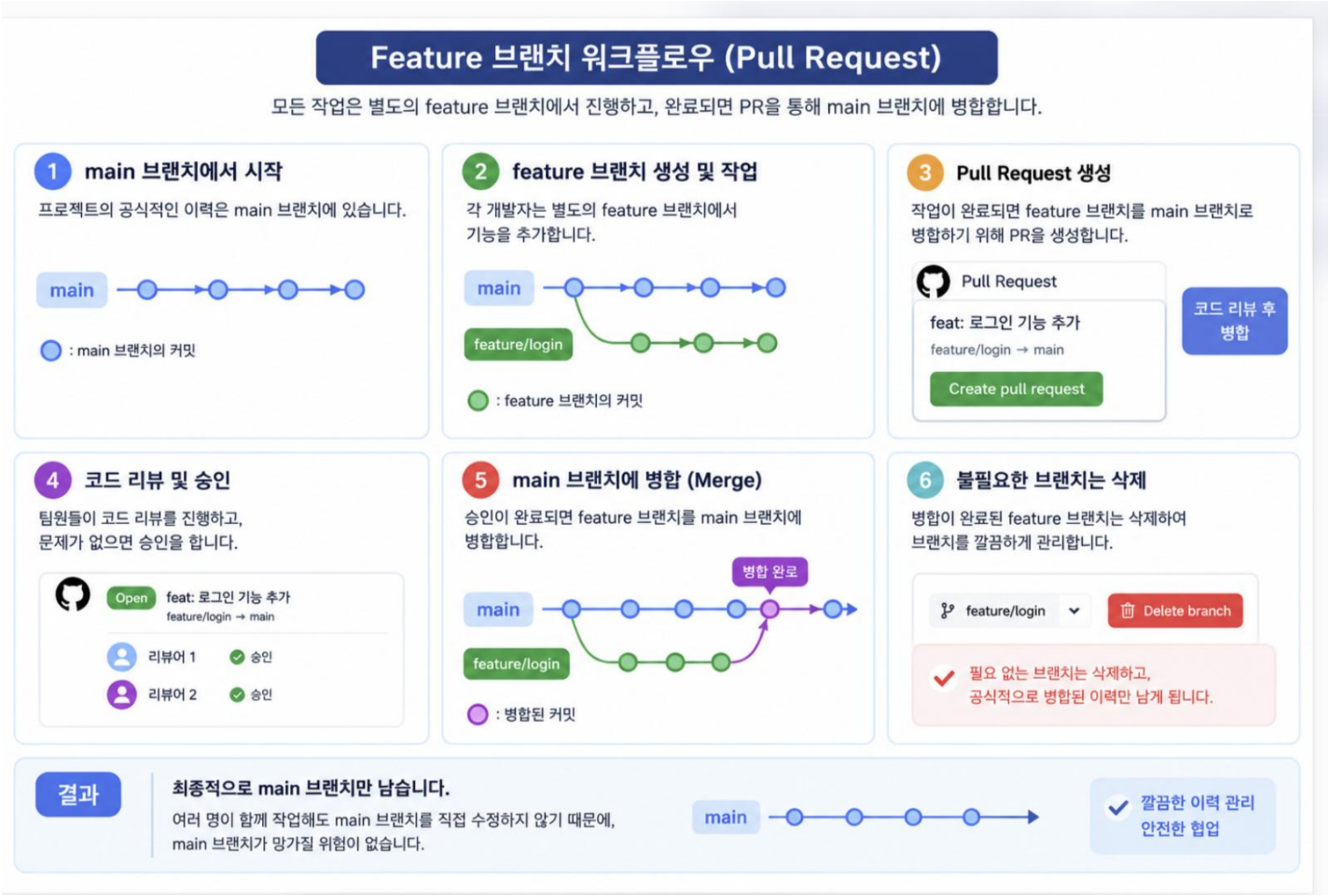


git 협업 워크 플로우

1. 협업 워크플로우

개요

프로젝트 구축할 때 협업 개발자와 함께 코드 관리를 github을 활용하여 관리하는 방법.



2. Feature 브랜치 워크플로우 병합

PR (Pull Request)

누군가는 feature 브랜치에 반영된 변경사항을 main 브랜치와 병합해야 된다.

이때 누구든지 모두가 병합하도록 하는 것 보다는 특히 조직에서 절대적으로 코드를 무결하게 유지해야 하거나 개발한 S/W 사용자를 고려해야 할 수도 있는데, 누군가가 엉망인 코드를 main 브랜치에 커밋했을 때 큰일이 발생될 수 있다.

결국 우리나 다른 누군가가 병합하더라도 병합을 해도 되는지 확인 하고 승인 받는 절차가 필요하다.
누군가의 요청으로 병합을 하게 됐는지 상관없이 PR를 이용하면 코드 리뷰 단계를 거치게 된다.

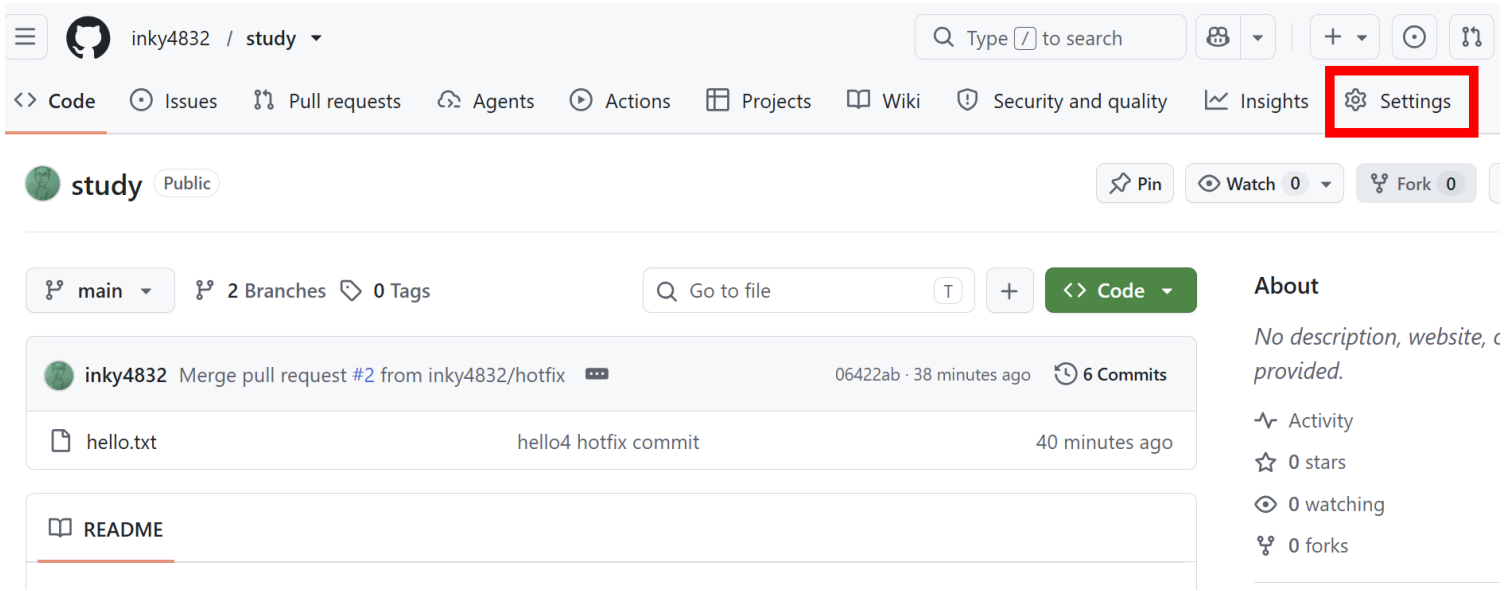
즉 PR은 개발자가 새로운 작업을 브랜치에 올렸을 때 팀원이나 공동 작업자들에게 검토를 요청하고 병합을 승인 또는 반려해달라고 요청하는 방법이다.

3. Feature 브랜치 워크플로우 실습

시나리오

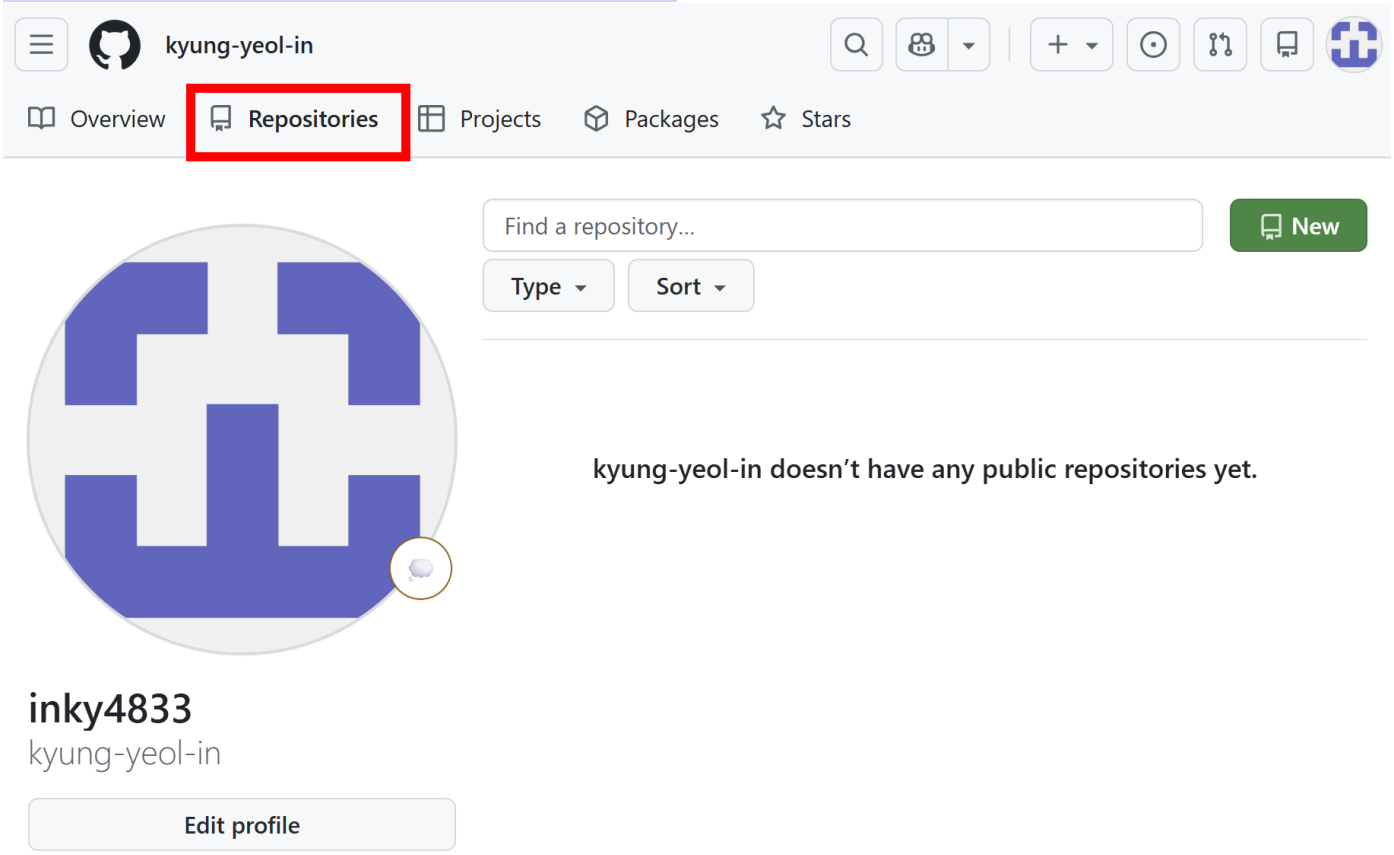
Github의 study 원격 저장소의 owner 인 inky4832@daum.net 가 협업자로 inky4833@gmail.com 을 등록하여 프로젝트를 구축하는 실습이다.

inky4832 owner github 화면



3. Feature 브랜치 워크플로우 실습

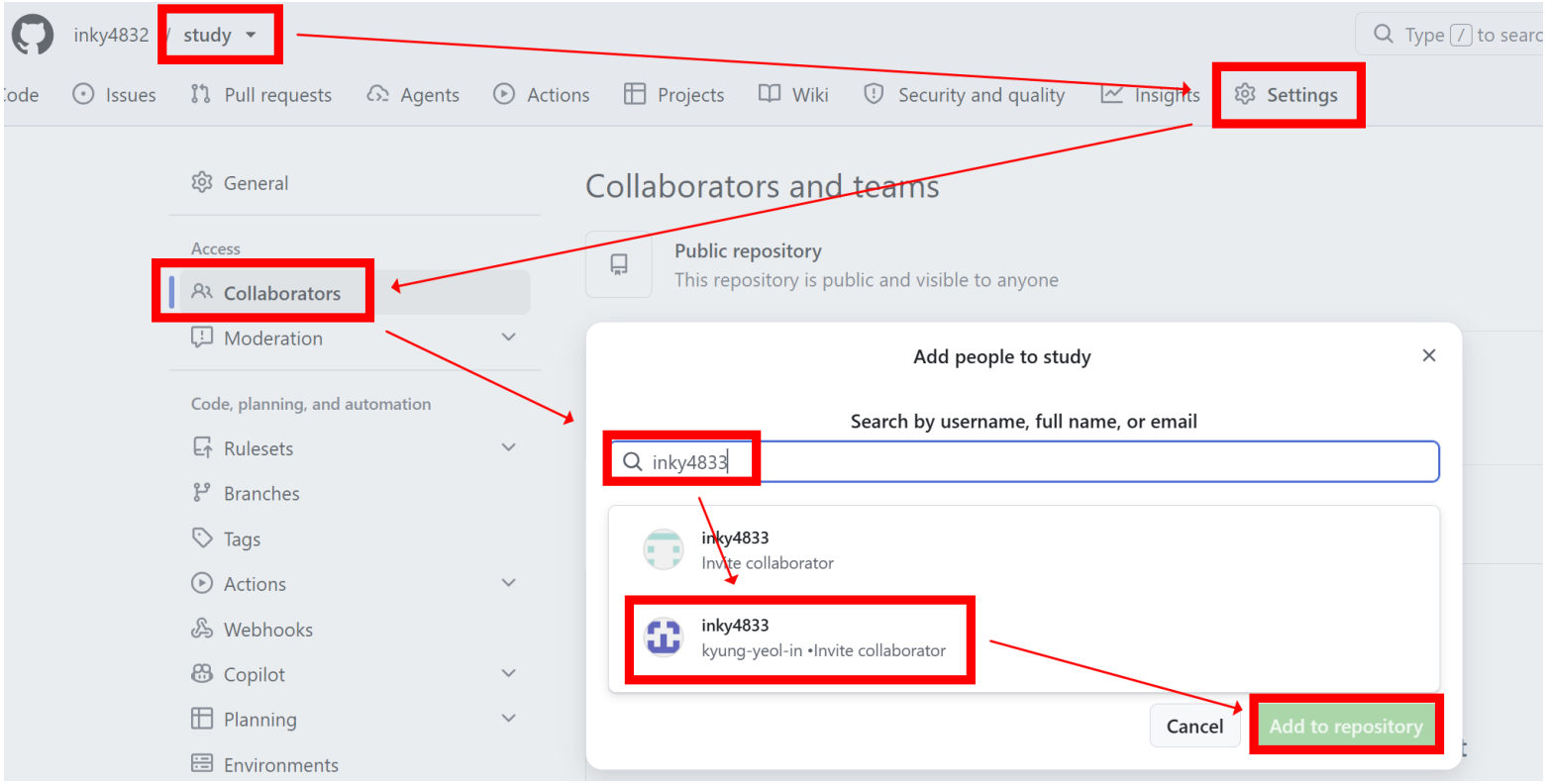
inky4833 협업자 github 화면



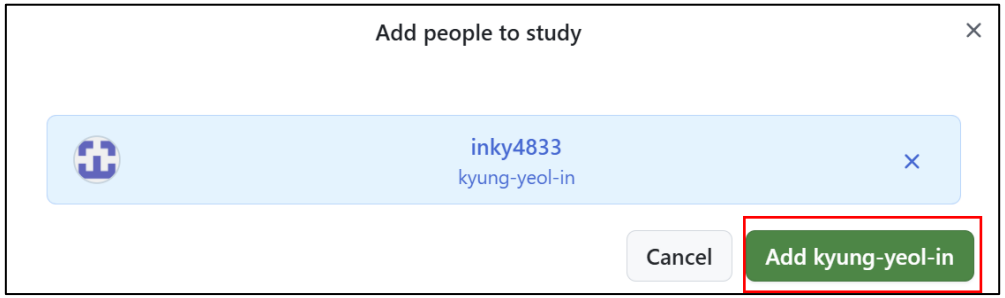
3. Feature 브랜치 워크플로우 실습

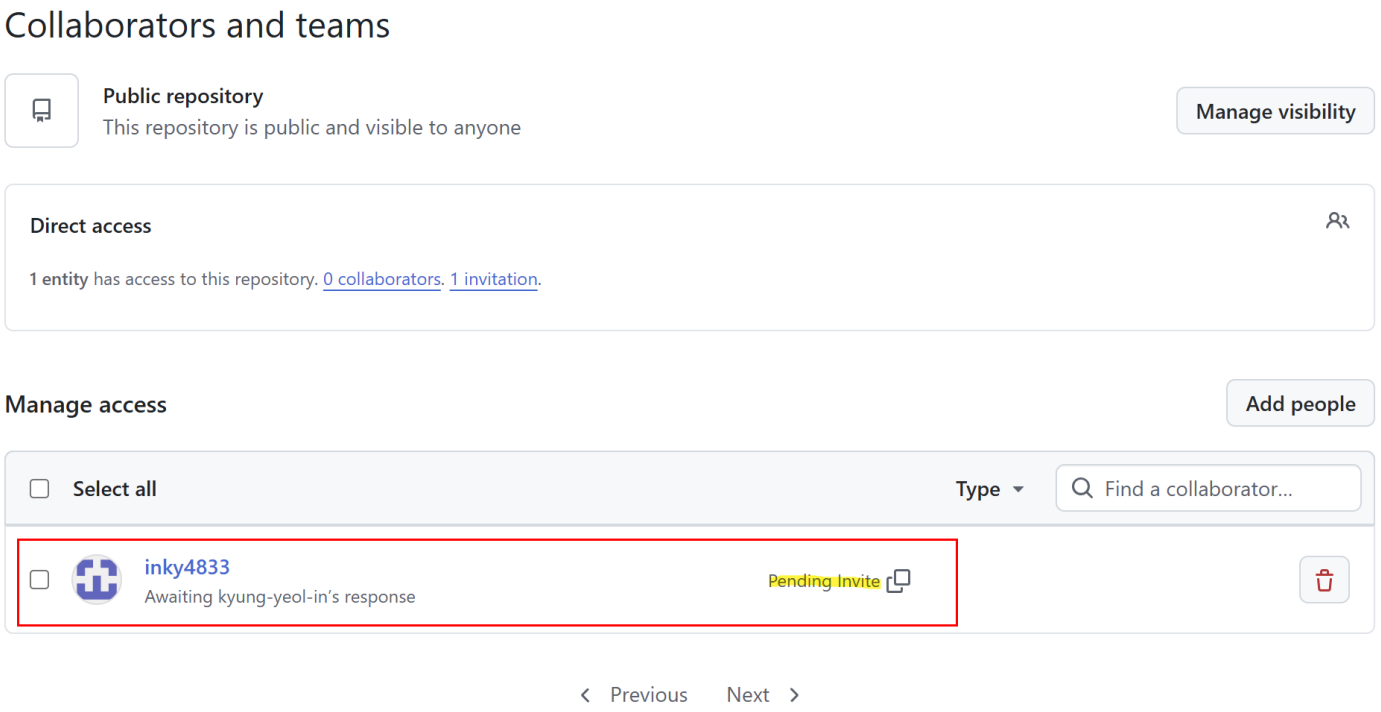
1) owner 인 inky4832가 github 에서 inky4833 을 협력자로 등록

study 저장소 > Settings > Collaborators > Add people 버튼 > inky4833 github 계정 추가 > inky4833@gmail.com 으로 메일 전송됨 > 메일 확인 후 View invitation 버튼 클릭 > 협력자 github 에서 Accept invitation 버튼 클릭 > 협력자 github에 study 저장소가 생성됨. (Settings 링크 없음)



3. Feature 브랜치 워크플로우 실습





3. Feature 브랜치 워크플로우 실습

2) inky4833 협업자 메일 확인



@inky4832 has invited you to collaborate on the inky4832/study repository

You can accept or decline this invitation. You can also head over to https://github.com/inky4832/study to check out the repository or visit @inky4832 to learn a bit more about them.

This invitation will expire in 7 days.

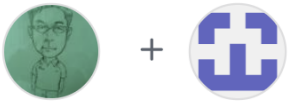
View invitation

Note: This invitation was intended for inky4833@gmail.com. If you were not expecting this invitation, you can ignore this email. If @inky4832 is sending you too many emails, you can block them or report abuse.

Getting a 404 error? Make sure you're signed in as **kyung-yeol-in**.

Button not working? Copy and paste this link into your browser:

3) inky4833 협업자 github 화면



inky4832 invited you to collaborate on inky4832/study

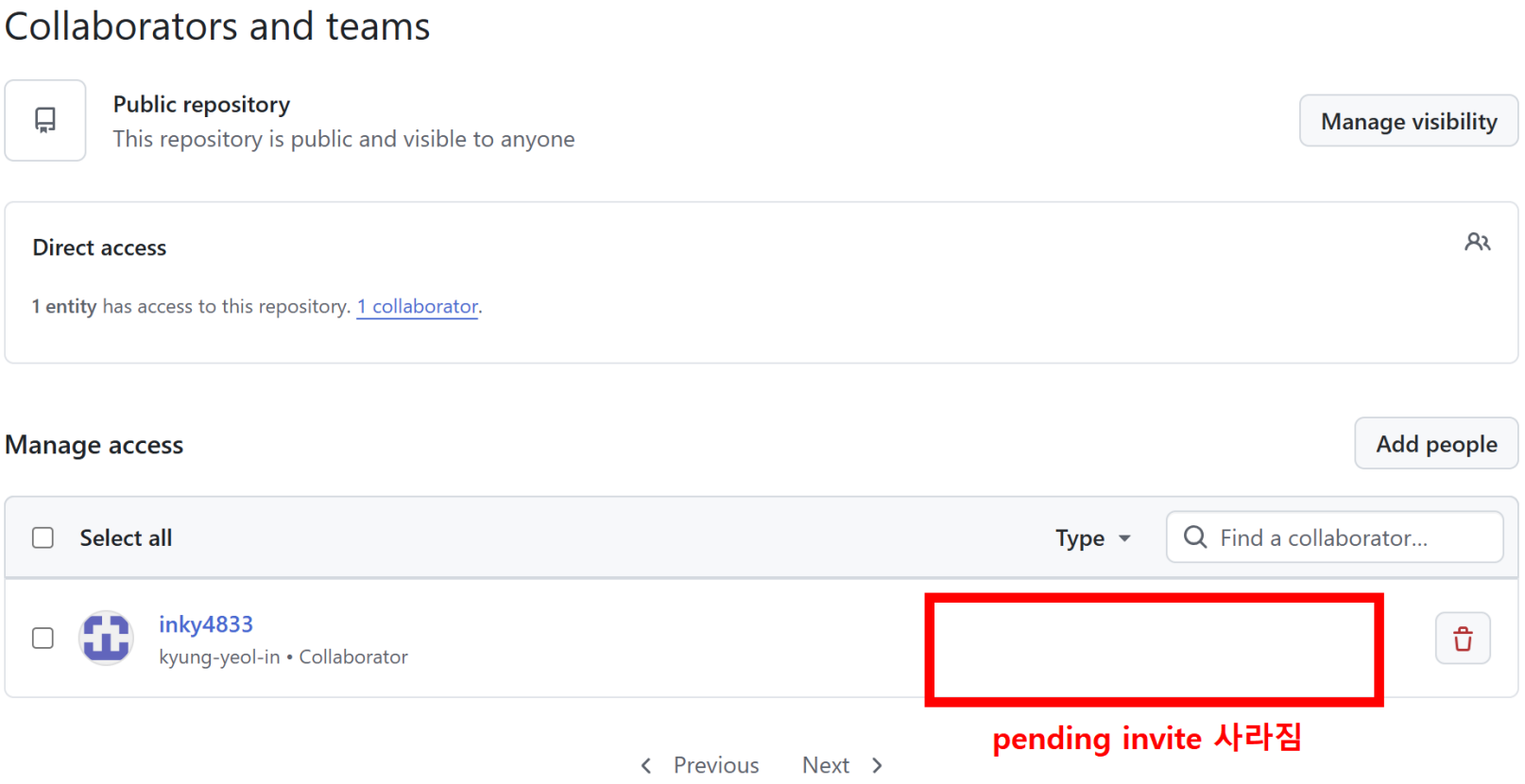
Accept invitation | Decline invitation

Owners of study will be able to see:

- Your public profile information
- Certain activity within this repository
- Country of request origin
- Your access level for this repository
- Your IP address

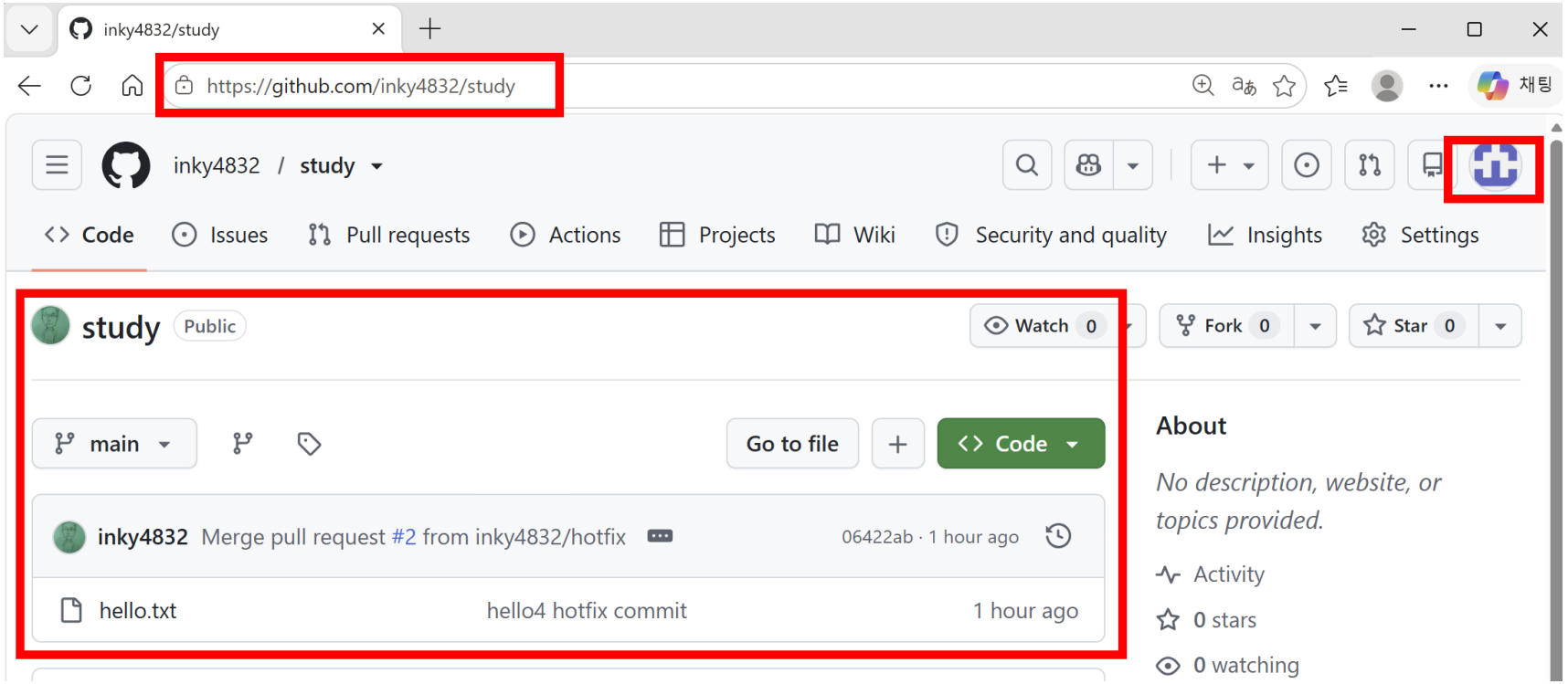
Is this user sending spam or malicious content?
Block inky4832 · Report abuse

4)inky4832 owner github 화면

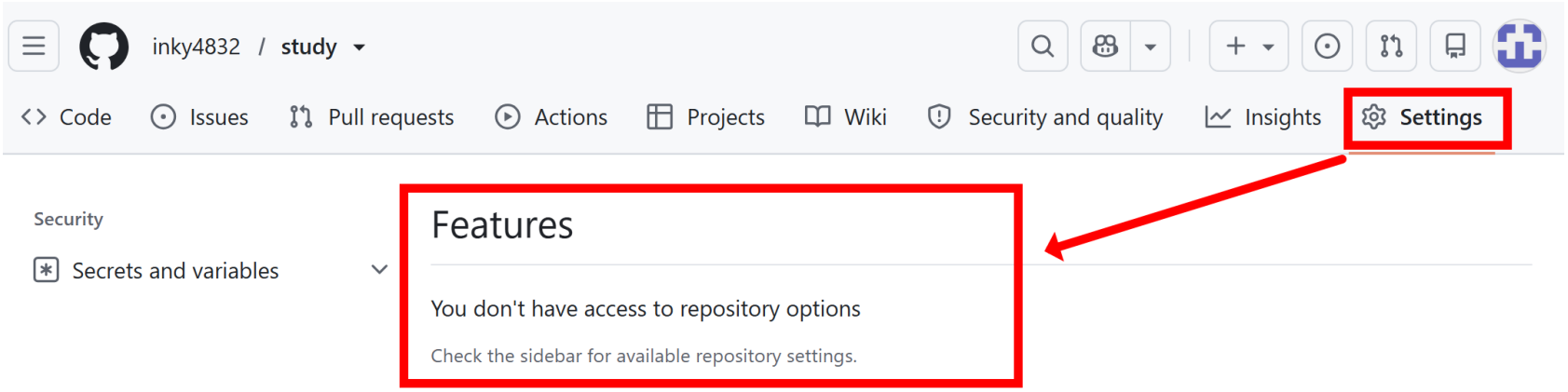


3. Feature 브랜치 워크플로우 실습

5) inky4833 협업자 github 화면



3. Feature 브랜치 워크플로우 실습



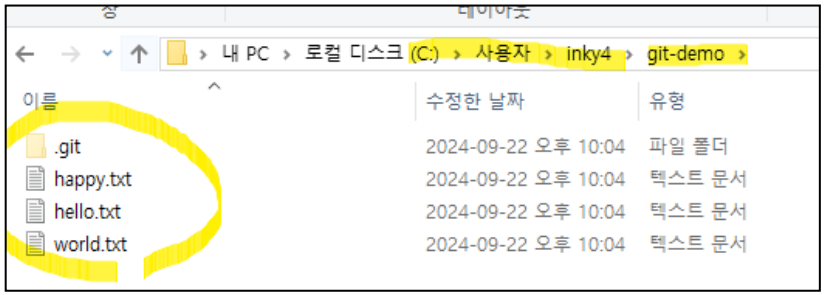
3. Feature 브랜치 워크플로우 실습

6) 협력자 PC에서 git clone 으로 inky4832 owner 원격 저장소를 복제

```
inky4@DESKTOP-KERHMJL MINGW64 ~
$ pwd
/c/Users/inky4

inky4@DESKTOP-KERHMJL MINGW64 ~
$ git clone https://github.com/inky4832/study.git git-demo
Cloning into 'git-demo'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 9 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.
```

git-demo 디렉터리가 생성되고
git init 까지 실행됨



3. Feature 브랜치 워크플로우 실습

7) 협력자 PC에서 원격 저장소 정보 보기

```
inky4@DESKTOP-KERHMJL MINGW64 ~
$ cd git-demo/

inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ git remote -v
origin  https://github.com/inky4832/study.git (fetch)
origin  https://github.com/inky4832/study.git (push)

inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ git remote get-url origin
https://github.com/inky4832/study.git
```

8) 협력자 PC에서 feature 브랜치 생성후 작업 추가 및 커밋

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ git branch feature        feature 브랜치 생성

inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ git switch feature        feature 브랜치로 이동
Switched to branch 'feature'

inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (feature)
$ touch study.txt           study.txt 파일 생성
```

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (feature)
$ git add study.txt

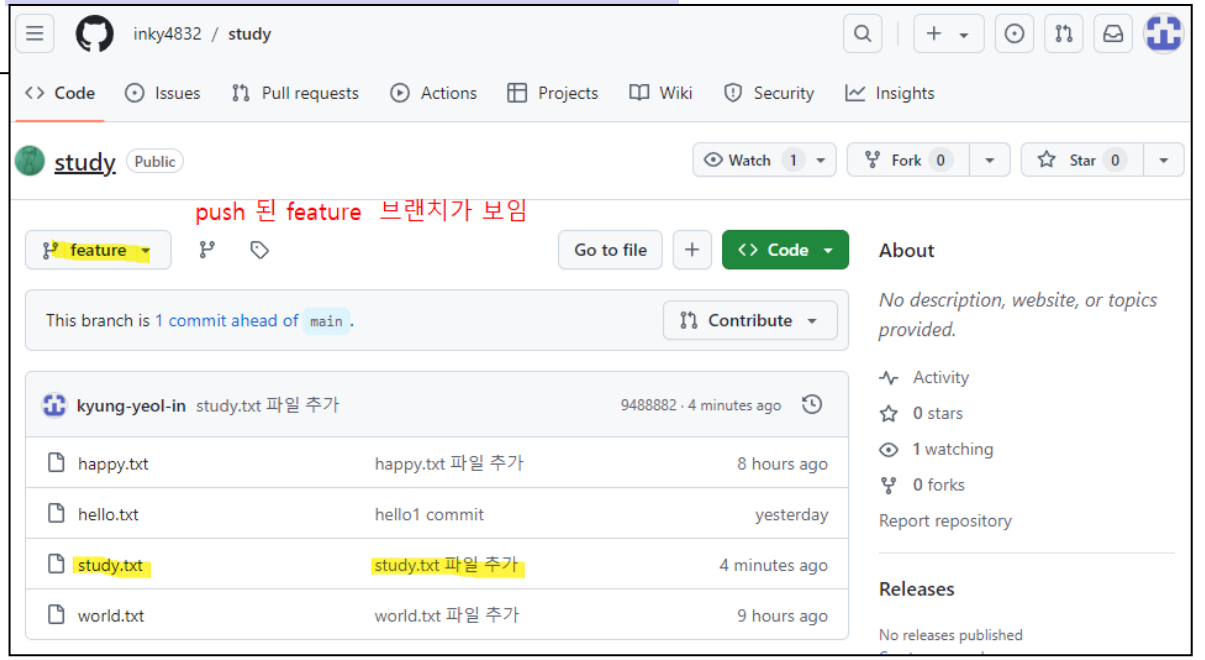
inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (feature)
$ git commit -m "study.txt 파 일  추 가 "
[feature 9488882] study.txt 파 일  추 가
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 study.txt
```

3. Feature 브랜치 워크플로우 실습

9) 협력자 PC에서 inky4832 owner 원격 저장소에 push

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (feature)
$ git push origin feature
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 351 bytes | 351.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature' on GitHub by visiting:
remote:      https://github.com/inky4832/s
remote:
To https://github.com/inky4832/study.git
 * [new branch]      feature -> feature
```

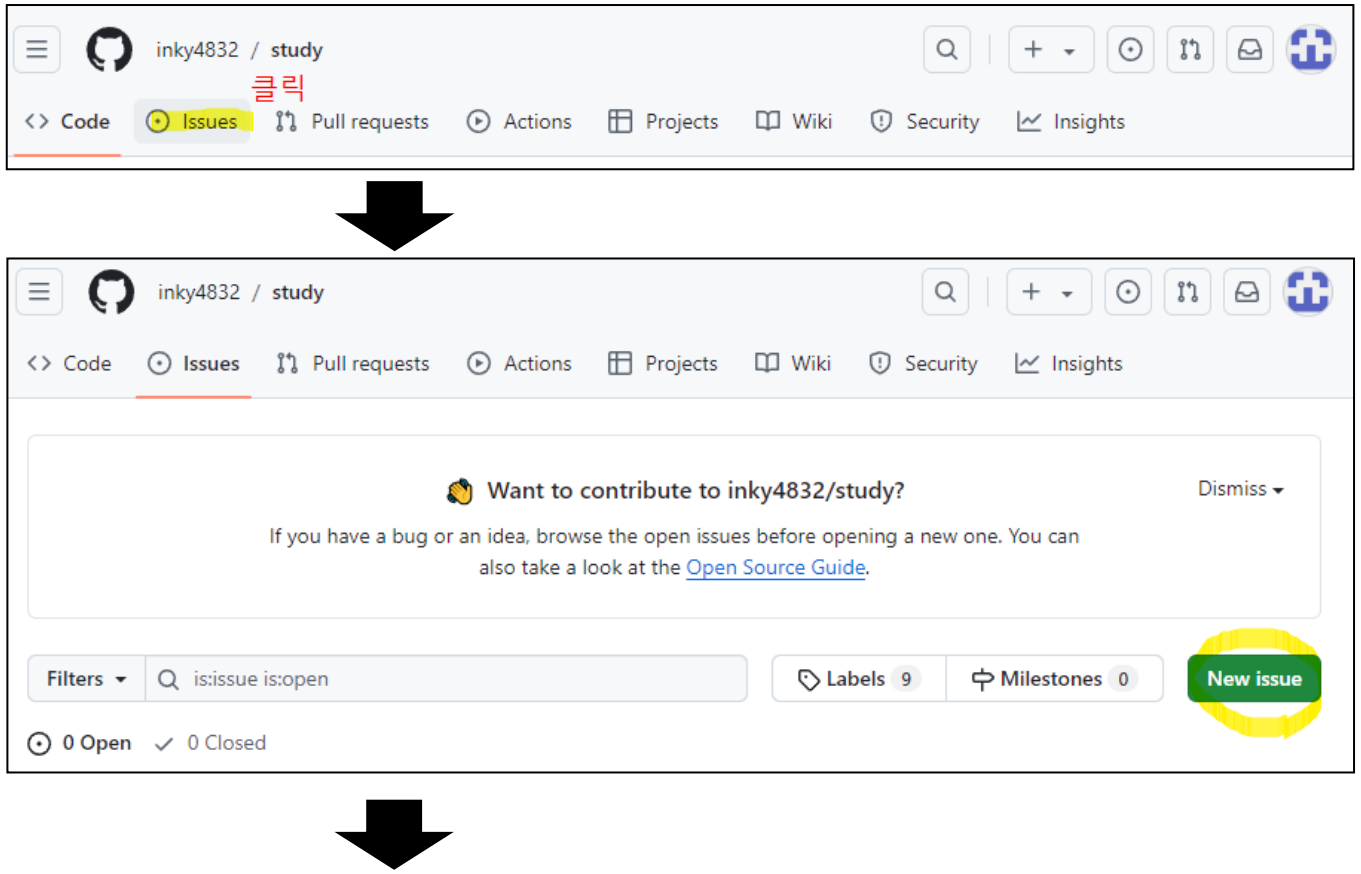
inky4833 협력자 github 화면



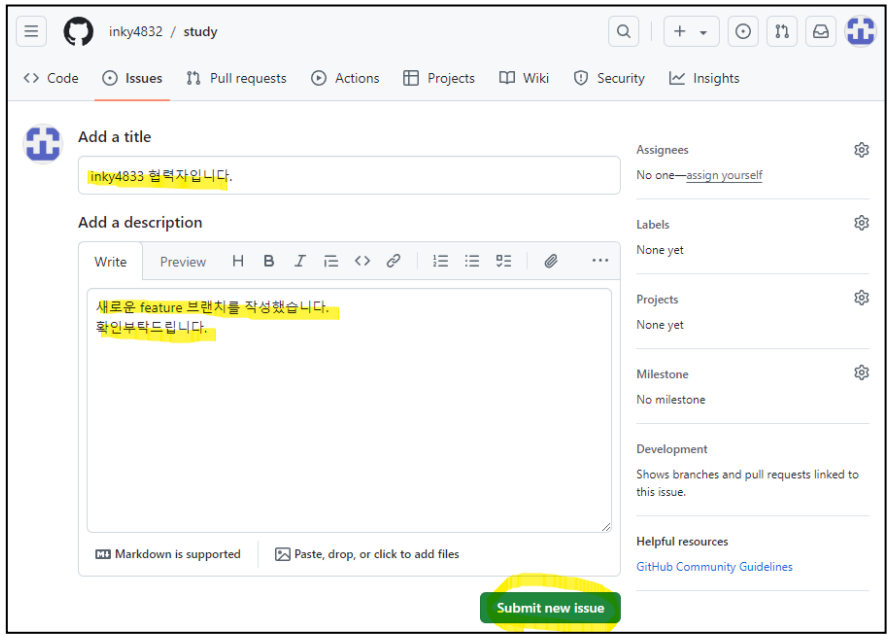
3. Feature 브랜치 워크플로우 실습

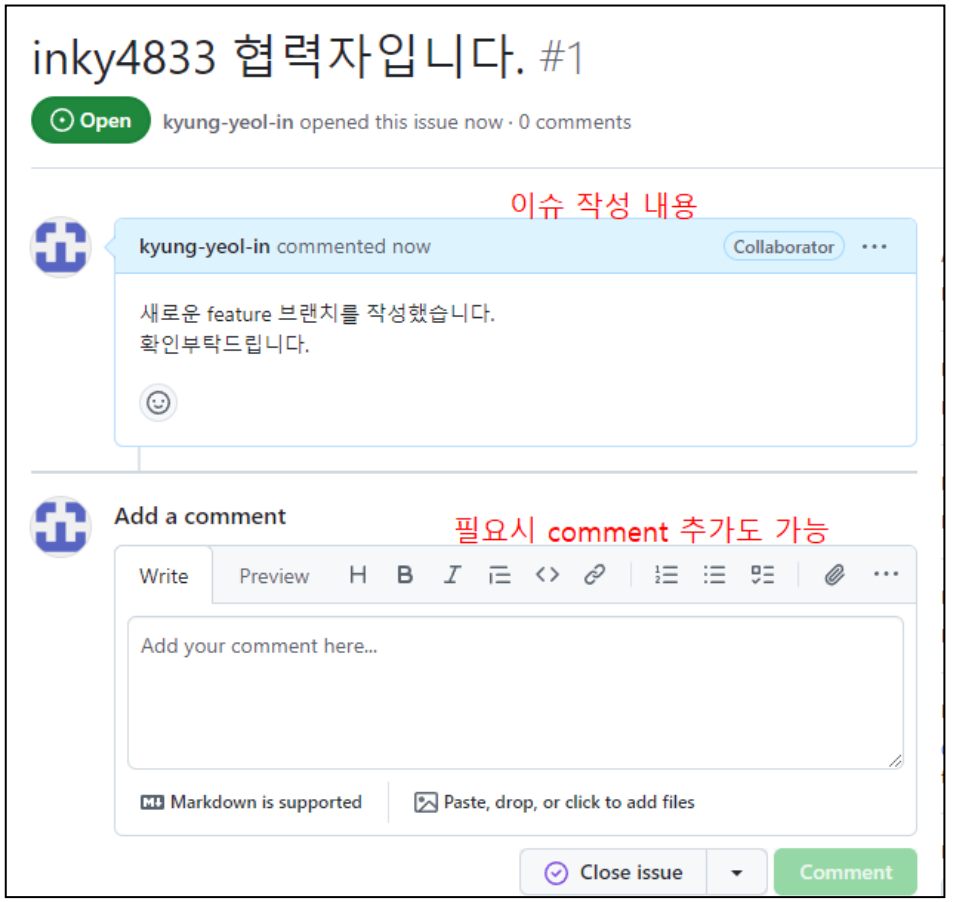
10) 협력자 github에서 특정 이슈를 inky4832 owner에게 보내기 (옵션 기능)

inky4833 협력자 github 화면



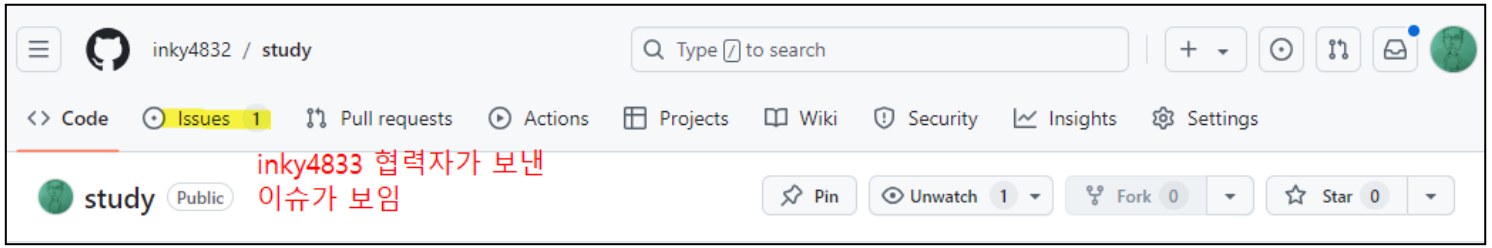
3. Feature 브랜치 워크플로우 실습





3. Feature 브랜치 워크플로우 실습

inky4832 owner github 화면



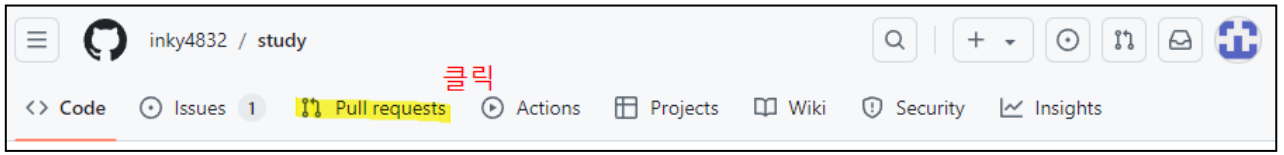


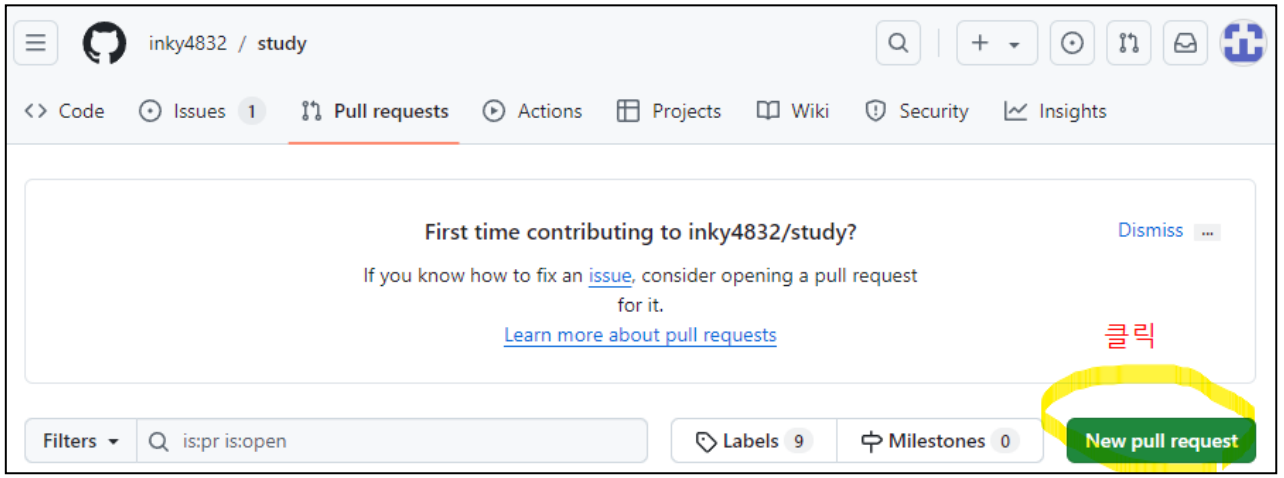


3. Feature 브랜치 워크플로우 실습

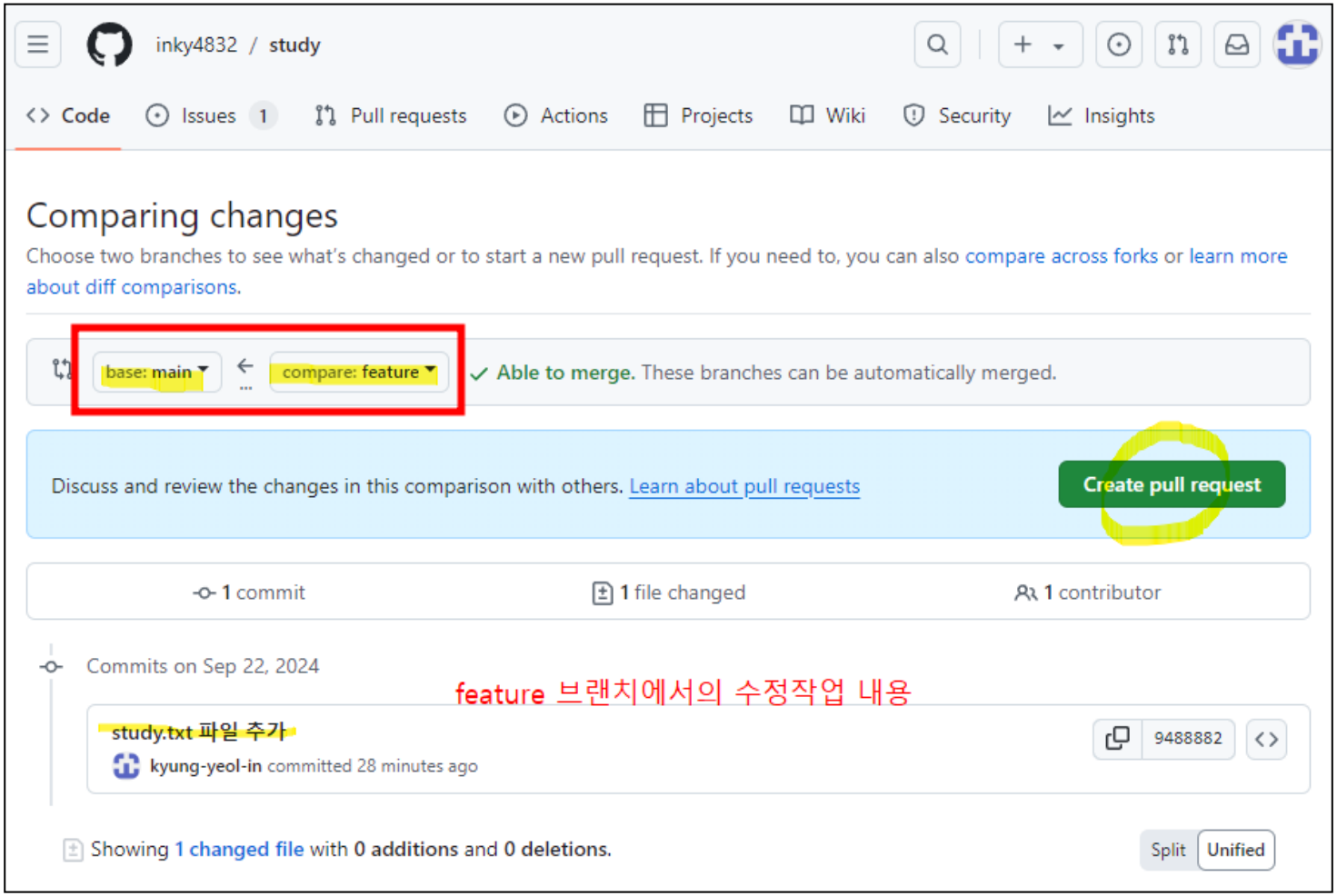
11) 협력자 github에서 feature 브랜치를 병합하도록 inky4832 owner에게 PR 요청하기

inky4833 협력자 github 화면

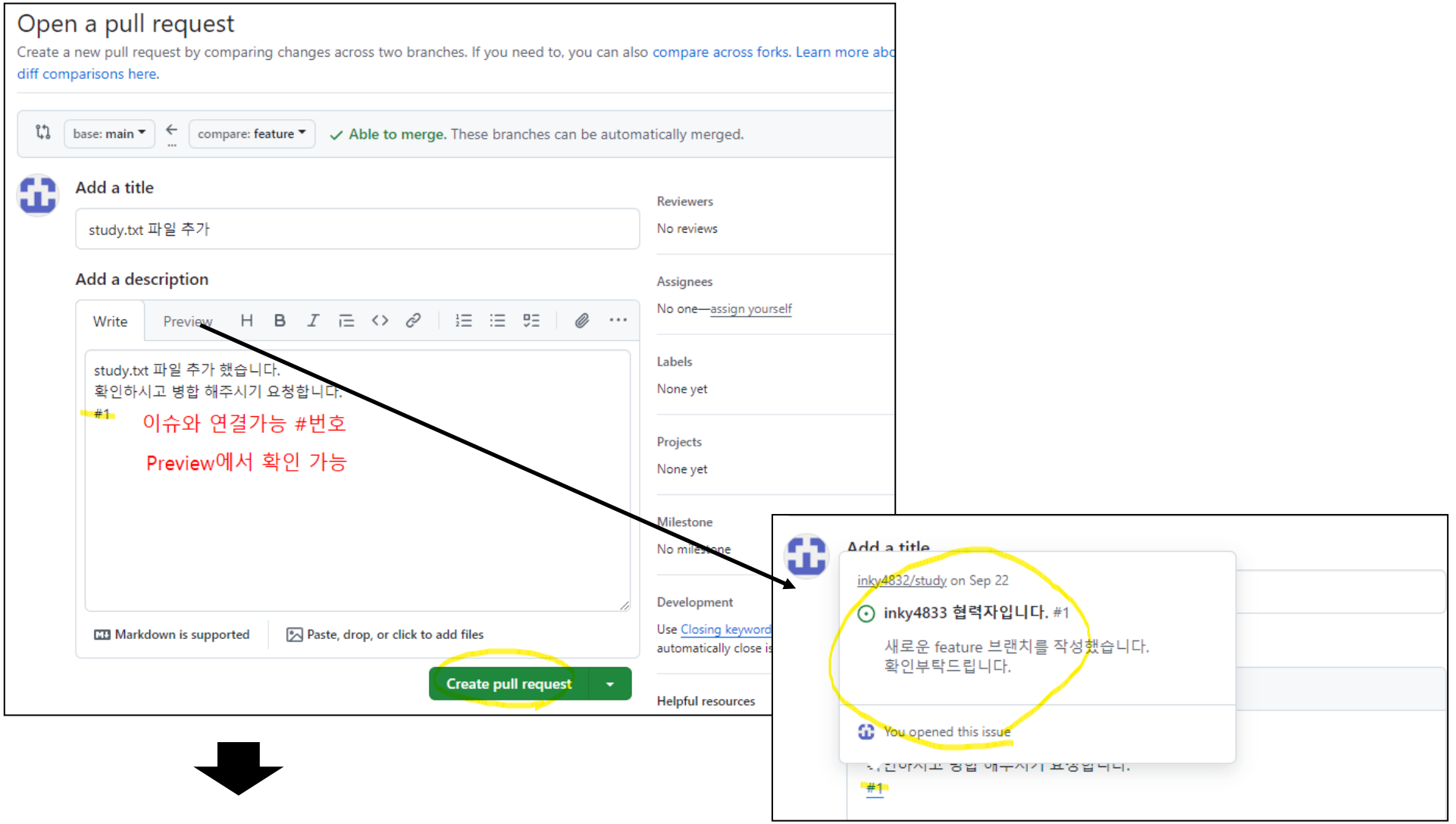




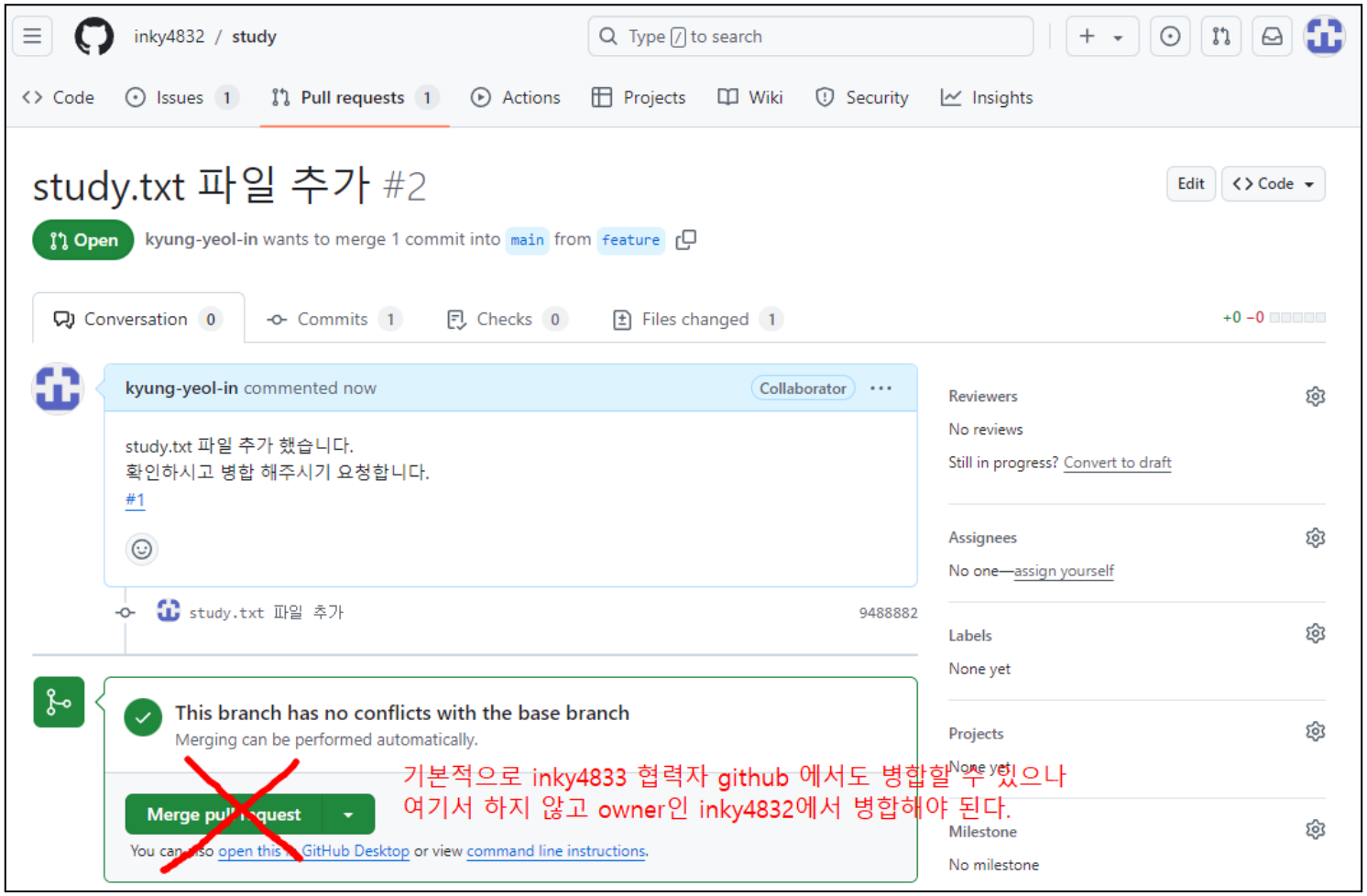
3. Feature 브랜치 워크플로우 실습



3. Feature 브랜치 워크플로우 실습



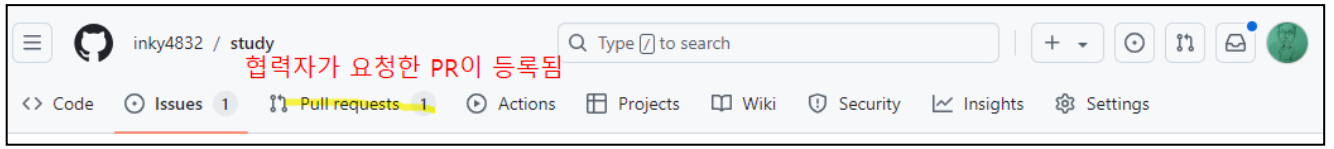
3. Feature 브랜치 워크플로우 실습

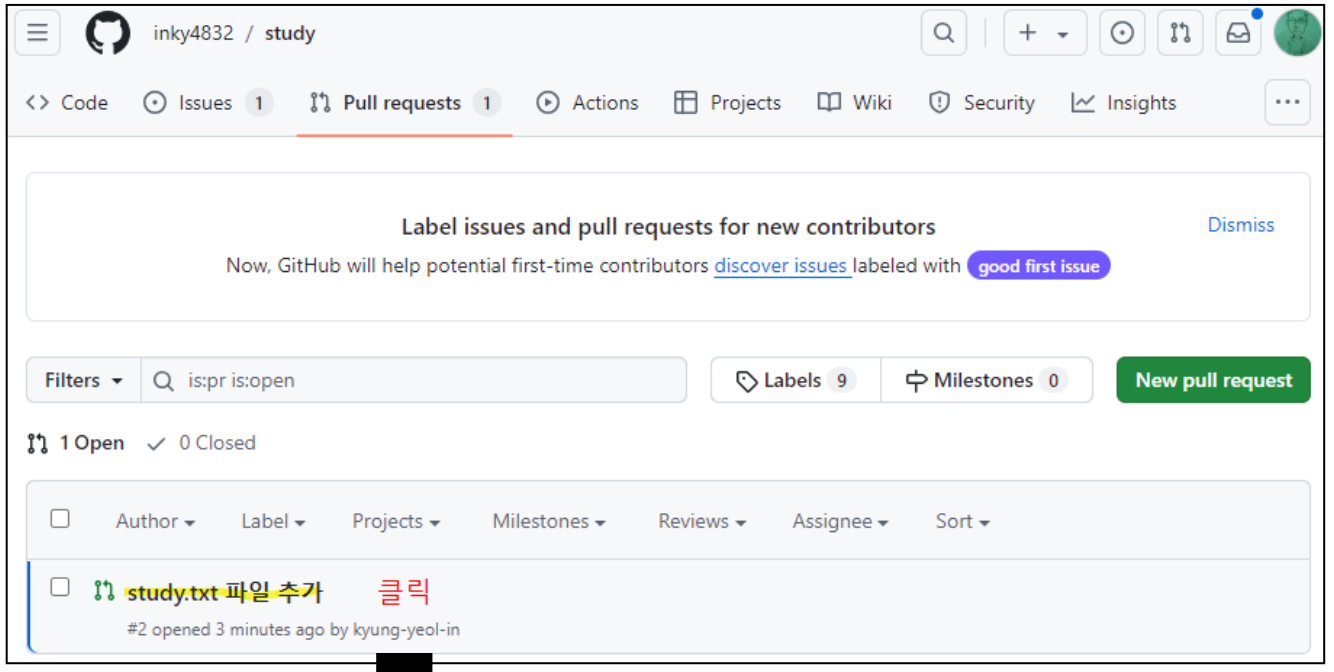


3. Feature 브랜치 워크플로우 실습

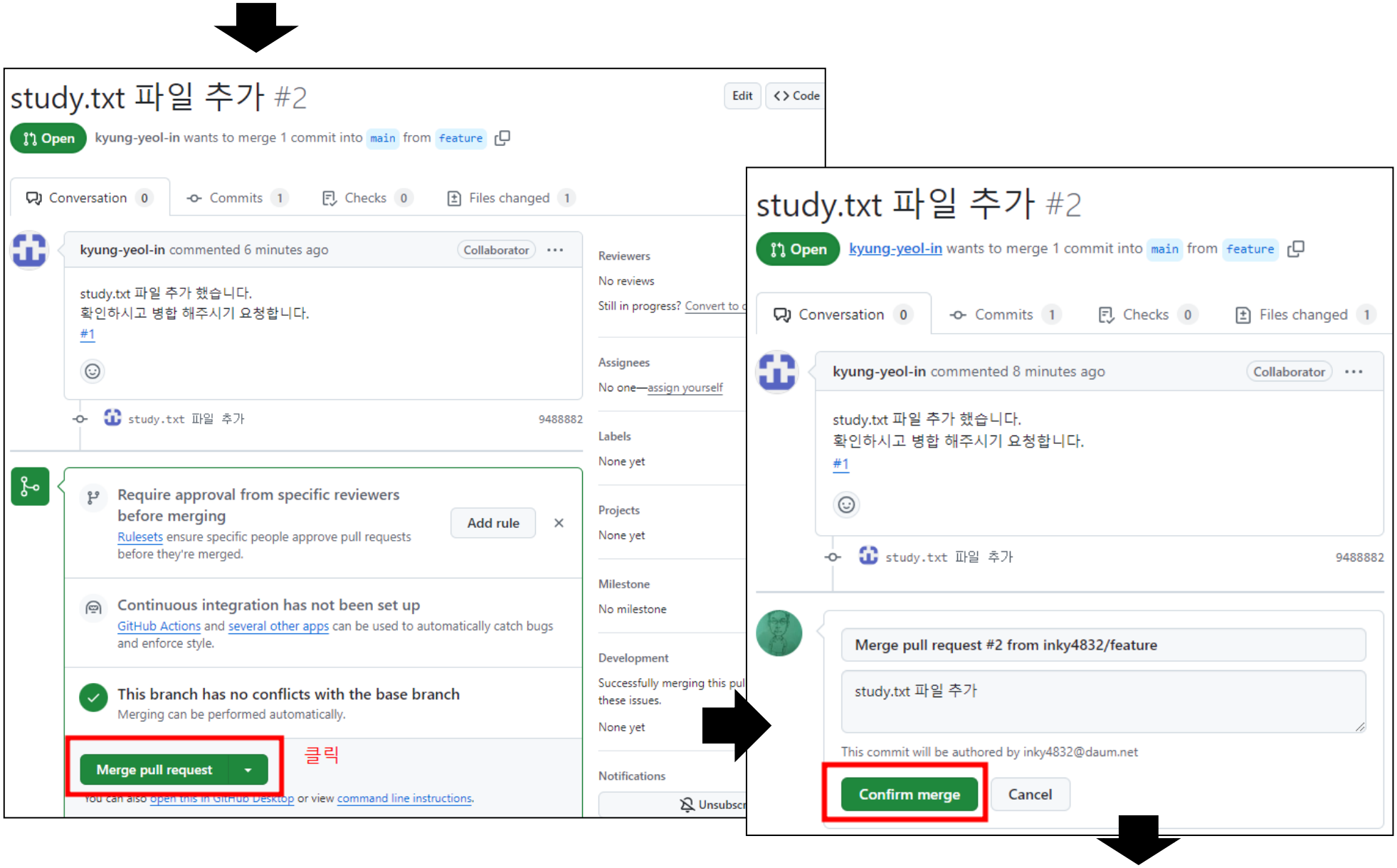
12) inky4832 owner github 에서 PR 요청 처리하기

inky4832 owner github 화면

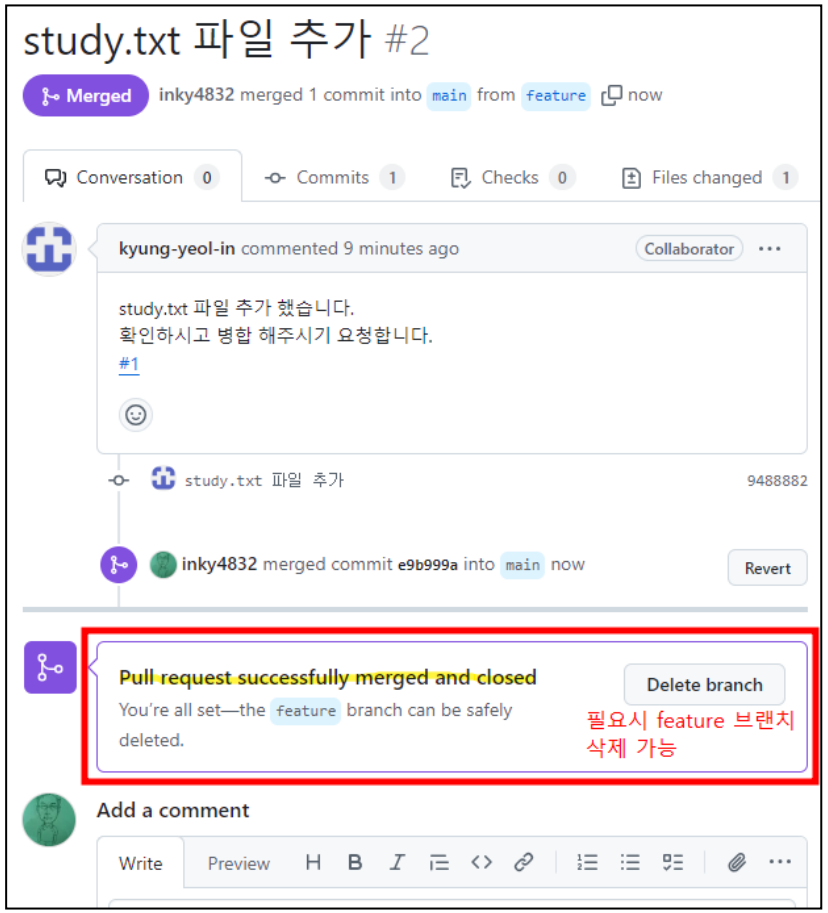




3. Feature 브랜치 워크플로우 실습



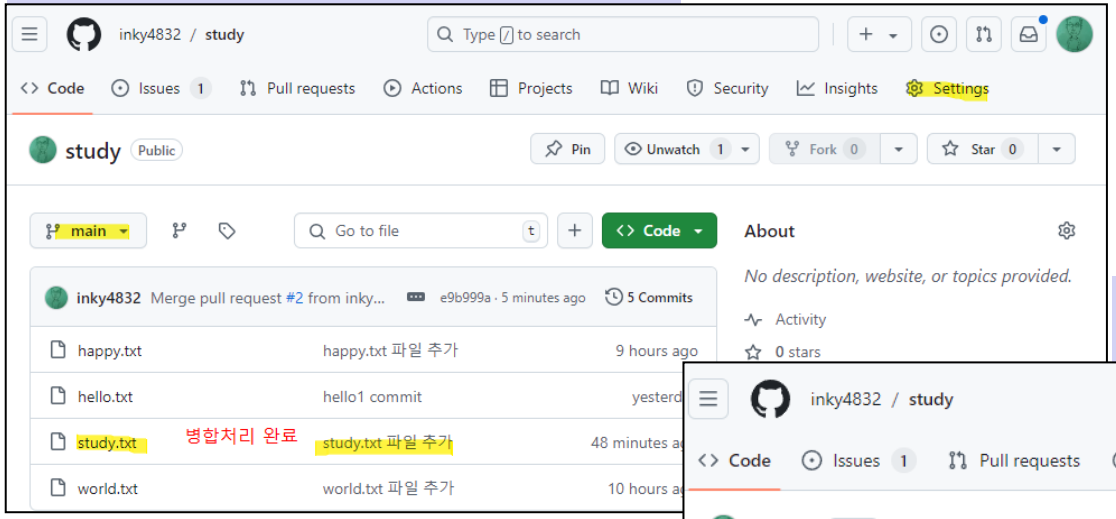
3. Feature 브랜치 워크플로우 실습



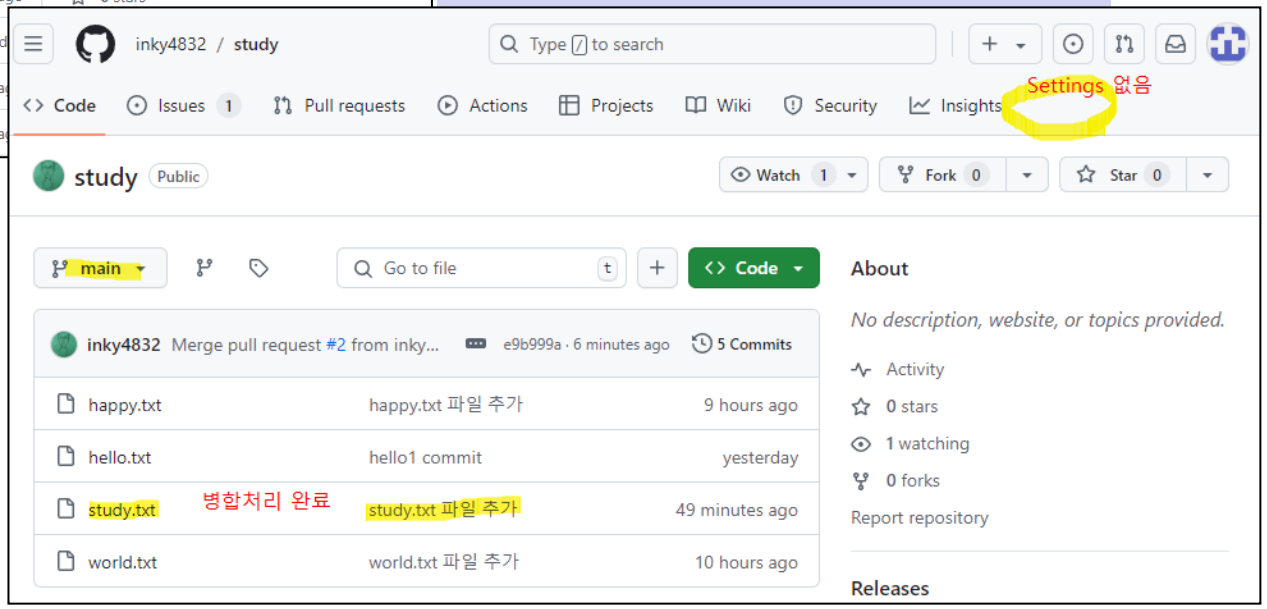
3. Feature 브랜치 워크플로우 실습

13) inky4832 owner 및 inky4833 협력자 github 에서 PR 처리 확인하기

inky4832 owner github 화면



inky4833 협력자 github 화면



14) 원격 저장소 최신 커밋을 로컬 저장소에 반영

git pull 작업후 로컬 저장소 목록

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack
Unpacking objects: 100% (1/1), 915 bytes | 305.00 K
From https://github.com/inky4832/study
 * branch            main       -> FETCH_HEAD
   af16fc7..e9b999a  main       -> origin/main
Updating af16fc7..e9b999a
Fast-forward
 study.txt | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 study.txt
```

Thank you